

Bab 10. Model, Database, dan CRUD di Laravel

10.1. Tujuan

Materi ini merupakan pengembangan dari aplikasi yang telah dibangun pada bab sebelumnya. Jika pada bab sebelumnya aplikasi hanya menampilkan halaman dashboard sederhana setelah login berhasil, pada bab ini dashboard tersebut dikembangkan menjadi sistem manajemen konten (CMS) yang lengkap. Pembahasan mencakup konsep migrasi database dan Eloquent ORM sebagai fondasi pengelolaan data di Laravel, termasuk pendefinisian relasi antar tabel menggunakan Eloquent Relationship. Operasi CRUD diimplementasikan untuk tiga entitas sekaligus yaitu artikel, penulis, dan kategori artikel menggunakan Resource Controller dan RESTful Routing. Tampilan dibangun menggunakan Bootstrap 5 dengan layout yang konsisten di seluruh halaman melalui Blade layout. Upload gambar ditangani menggunakan Laravel Storage dengan folder terpisah untuk setiap entitas. Seluruh operasi dilindungi dengan middleware autentikasi yang telah dibangun pada bab sebelumnya sehingga hanya pengguna yang sudah login yang dapat mengakses dan mengelola data.

Setelah mempelajari materi ini, diharapkan mampu:

- a. Menjelaskan konsep migrasi database di Laravel dan perannya dalam pengelolaan struktur database.
- b. Menggunakan Eloquent ORM untuk operasi CRUD dan mendefinisikan relasi antar tabel menggunakan Eloquent Relationship.
- c. Membuat Resource Controller dan mendefinisikan RESTful Route untuk mengelola tiga entitas sekaligus.
- d. Membangun layout Blade yang konsisten dan dapat diwarisi oleh seluruh View dalam aplikasi.
- e. Mengimplementasikan operasi CRUD lengkap untuk entitas artikel, penulis, dan kategori artikel menggunakan form submission dan PRG pattern.
- f. Menangani upload gambar menggunakan Laravel Storage dengan folder terpisah untuk setiap entitas.
- g. Menampilkan notifikasi hasil operasi menggunakan session flash message.
- h. Menerapkan validasi input di sisi server menggunakan fitur validasi bawaan Laravel.

10.2 Dasar Teori

Sebelum ke tahap implementasi, perlu dipahami beberapa konsep yang mendasari pembahasan pada bab ini. Pertama adalah migrasi database sebagai mekanisme pengelolaan struktur database menggunakan kode PHP. Selanjutnya dibahas Eloquent ORM sebagai antarmuka utama untuk berinteraksi dengan database, termasuk pendefinisian relasi antar tabel menggunakan Eloquent Relationship. Pembahasan dilanjutkan dengan Resource Controller dan RESTful Routing sebagai pola standar Laravel untuk membangun fitur CRUD, Blade template engine untuk membangun tampilan yang konsisten, serta Laravel Storage untuk pengelolaan file upload. Bagian akhir

membahas validasi input bawaan Laravel dan PRG pattern beserta session flash message sebagai mekanisme notifikasi hasil operasi.

10.2.1 Migrasi Database di Laravel

Migrasi adalah mekanisme pengelolaan struktur database menggunakan kode PHP. Dengan migrasi, perubahan struktur database seperti membuat tabel, menambah kolom, atau mengubah tipe data didefinisikan dalam file PHP yang dapat dijalankan, dibatalkan, dan direproduksi kapan saja. Pendekatan ini memungkinkan struktur database dikelola bersama kode aplikasi dalam satu repositori.

Pada pendekatan yang telah digunakan di bab-bab sebelumnya, struktur database dibuat dengan menulis perintah SQL secara langsung dan menjalankannya melalui phpMyAdmin. Pendekatan ini sudah cukup untuk aplikasi sederhana, namun seiring bertambahnya kompleksitas aplikasi, pengelolaan perubahan struktur database secara manual menjadi semakin tidak terstruktur. Migrasi Laravel hadir sebagai solusi dengan mendefinisikan struktur database dalam kode PHP yang terorganisir dan dapat dijalankan ulang kapan saja.

Laravel menyimpan file migrasi di folder `database/migrations/`. Setiap file migrasi berisi dua method utama:

- up()** mendefinisikan perubahan yang akan diterapkan ke database, misalnya membuat tabel baru atau menambah kolom.
- down()** mendefinisikan cara membatalkan perubahan yang dilakukan oleh method `up()`, misalnya menghapus tabel atau kolom yang baru ditambahkan.

Berikut contoh file migrasi untuk membuat tabel `kategori_artikel`:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('kategori_artikel', function (Blueprint $table) {
            $table->id();
            $table->string('nama_kategori', 100)->unique();
            $table->text('keterangan')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('kategori_artikel');
    }
};
```

Migrasi dijalankan melalui terminal menggunakan perintah Artisan:

```
php artisan migrate
```

Untuk membatalkan migrasi terakhir:

```
php artisan migrate:rollback
```

Perlu diperhatikan bahwa pada bab ini tabel-tabel yang dibutuhkan sudah tersedia di database `db_blog` dari bab sebelumnya sehingga migrasi tidak perlu dijalankan ulang. Konsep migrasi tetap penting dipahami karena menjadi fondasi pengelolaan database di setiap proyek Laravel baru.

10.2.2 Eloquent ORM

Eloquent adalah ORM (Object-Relational Mapping) bawaan Laravel yang menyediakan antarmuka untuk berinteraksi dengan database menggunakan sintaks PHP yang ekspresif tanpa perlu menulis query SQL secara langsung. Setiap tabel database direpresentasikan oleh sebuah kelas Model, dan setiap baris data direpresentasikan sebagai instance dari kelas tersebut.

Pada pendekatan yang telah digunakan di bab-bab sebelumnya, query database ditulis menggunakan SQL mentah melalui `mysqli` dan prepared statements. Eloquent menawarkan cara yang lebih ringkas dan lebih mudah dibaca untuk melakukan operasi yang sama.

Berikut perbandingan query menggunakan `mysqli` dengan Eloquent:

Tabel 10.1 Perbandingan query `mysqli` dengan Eloquent

Operasi	mysqli	Eloquent (Laravel)
Ambil semua data	<code>\$stmt = \$koneksi->prepare("SELECT * FROM kontak"); \$stmt->execute();</code>	<code>Kontak::all()</code>
Ambil data by id	<code>\$stmt = \$koneksi->prepare("SELECT * FROM kontak WHERE id = ?");</code>	<code>Kontak::find(\$id)</code>
Simpan data baru	<code>\$stmt = \$koneksi->prepare("INSERT INTO kontak ...");</code>	<code>Kontak::create(\$data)</code>
Perbarui data	<code>\$stmt = \$koneksi->prepare("UPDATE kontak SET ...");</code>	<code>\$kontak->update(\$data)</code>
Hapus data	<code>\$stmt = \$koneksi->prepare("DELETE FROM kontak WHERE id = ?");</code>	<code>\$kontak->delete()</code>

Setiap Model Eloquent secara konvensi merujuk ke tabel database yang namanya merupakan bentuk jamak dari nama Model. Model `Artikel` merujuk ke tabel `artikels`, Model `Penulis` merujuk ke tabel `penuliss`, dan seterusnya. Karena nama tabel yang digunakan pada bab ini tidak mengikuti konvensi tersebut, nama tabel perlu didefinisikan secara eksplisit di dalam Model menggunakan properti `$table`:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Artikel extends Model
{
    protected $table = 'artikel';

    protected $fillable = [
        'id_penulis',
        'id_kategori',
    ];
}
```

```

        'judul',
        'isi',
        'gambar',
        'hari_tanggal',
    ];
}

```

Properti `$fillable` mendefinisikan kolom mana saja yang boleh diisi secara massal melalui method `create()` atau `update()`. Kolom yang tidak terdaftar di `$fillable` tidak dapat diisi secara massal sehingga aplikasi terlindungi dari serangan mass assignment.

Eloquent juga menyediakan method pencarian data yang fleksibel. Berikut beberapa method yang sering digunakan:

- Artikel::all()** mengambil seluruh data dari tabel `artikel`.
- Artikel::find(\$id)** mengambil satu data berdasarkan primary key. Mengembalikan `null` jika data tidak ditemukan.
- Artikel::findOrFail(\$id)** mengambil satu data berdasarkan primary key. Melempar exception `ModelNotFoundException` jika data tidak ditemukan, yang secara otomatis menghasilkan response 404 di Laravel.
- Artikel::where('kolom', 'nilai')->get()** mengambil data dengan kondisi tertentu.
- Artikel::orderBy('kolom', 'desc')->get()** mengambil data dengan urutan tertentu.

10.2.3 Eloquent Relationship

Eloquent Relationship adalah fitur Eloquent yang memungkinkan pendefinisian relasi antar tabel langsung di dalam Model. Dengan relationship, data dari tabel yang berelasi dapat diakses sebagai properti objek tanpa perlu menulis query JOIN secara manual.

Pada bab ini terdapat dua jenis relasi yang digunakan berdasarkan struktur tabel `db_blog`:

- Tabel `penulis` berelasi ke tabel `artikel`, satu penulis dapat memiliki banyak artikel
- Tabel `kategori_artikel` berelasi ke tabel `artikel`, satu kategori dapat memiliki banyak artikel
- Tabel `artikel` berelasi ke tabel `penulis` dan `kategori_artikel`, setiap artikel dimiliki oleh satu penulis dan satu kategori

Dua jenis relationship yang digunakan adalah `hasMany` dan `belongsTo`. **hasMany** digunakan pada Model yang memiliki banyak data di tabel lain. Model `Penulis` menggunakan `hasMany` karena satu penulis dapat memiliki banyak artikel:

```

// Di Model Penulis
public function artikel()
{
    return $this->hasMany(Artikel::class, 'id_penulis');
}

```

belongsTo digunakan pada Model yang merujuk ke satu data di tabel lain. Model `Artikel` menggunakan `belongsTo` karena setiap artikel dimiliki oleh satu penulis dan satu kategori:

```

// Di Model Artikel

```

```

public function penulis()
{
    return $this->belongsTo(Penulis::class, 'id_penulis');
}

public function kategori()
{
    return $this->belongsTo(KategoriArtikel::class, 'id_kategori');
}

```

Setelah relationship didefinisikan, data dari tabel yang berelasi dapat diakses langsung sebagai properti objek di Controller maupun View:

```

// Mengambil seluruh artikel beserta data penulis dan kategori
$artikel = Artikel::with('penulis', 'kategori')->get();

// Mengakses data relasi di View
{{ $artikel->penulis->nama_depan }}
{{ $artikel->kategori->nama_kategori }}

```

Method `with()` pada contoh di atas disebut eager loading. Eager loading mengambil data relasi sekaligus dalam satu query tambahan sehingga lebih efisien dibandingkan mengambil data relasi satu per satu untuk setiap baris data. Tanpa eager loading, Laravel akan menjalankan query tambahan untuk setiap baris artikel yang ditampilkan. Kondisi ini dikenal sebagai masalah N+1 query yang dapat memperlambat aplikasi secara signifikan jika data yang ditampilkan berjumlah besar.

10.2.4 Resource Controller dan RESTful Routing

Resource Controller adalah jenis Controller di Laravel yang menyediakan tujuh method standar untuk menangani operasi CRUD secara lengkap. Ketujuh method tersebut mengikuti konvensi RESTful yang sudah mapan di industri sehingga struktur Controller menjadi konsisten dan mudah dipahami.

Berikut ketujuh method yang tersedia di Resource Controller beserta fungsinya:

Tabel 10.2 Method standar Resource Controller

Method	URL	HTTP Method	Fungsi
index	/artikel	GET	Menampilkan daftar seluruh artikel
create	/artikel/create	GET	Menampilkan form tambah artikel
store	/artikel	POST	Menyimpan artikel baru ke database
show	/artikel/{id}	GET	Menampilkan satu artikel
edit	/artikel/{id}/edit	GET	Menampilkan form edit artikel
update	/artikel/{id}	PUT/PATCH	Memperbarui artikel di database
destroy	/artikel/{id}	DELETE	Menghapus artikel dari database

Resource Controller dibuat menggunakan perintah Artisan dengan menambahkan flag `--resource`:

```
php artisan make:controller ArtikelController --resource
```

Laravel akan menghasilkan file Controller yang sudah berisi ketujuh method tersebut dalam keadaan kosong, siap diisi dengan logika bisnis.

Route untuk Resource Controller didefinisikan menggunakan method `Route::resource()` yang secara otomatis mendaftarkan ketujuh route sekaligus:

```
Route::resource('artikel', ArtikelController::class);
```

Satu baris tersebut setara dengan mendefinisikan tujuh Route secara manual. Untuk memverifikasi seluruh Route yang terdaftar, jalankan perintah berikut di terminal:

```
php artisan route:list
```

Pada bab ini tidak semua method digunakan. Method `show` tidak diimplementasikan karena aplikasi tidak memerlukan halaman detail untuk setiap entitas. Untuk membatasi method yang didaftarkan, gunakan method `only()`:

```
Route::resource('artikel', ArtikelController::class)->only([
    'index', 'create', 'store', 'edit', 'update', 'destroy'
]);
```

Perlu diperhatikan bahwa form HTML hanya mendukung method `GET` dan `POST`. Untuk method `PUT`, `PATCH`, dan `DELETE` yang dibutuhkan oleh Resource Controller, Laravel menyediakan directive Blade `@method()` yang menghasilkan hidden input berisi informasi method yang sebenarnya:

```
<form action="{{ route('artikel.update', $artikel->id) }}" method="POST">
    @csrf
    @method('PUT')
    ...
</form>
```

```
<form action="{{ route('artikel.destroy', $artikel->id) }}" method="POST">
    @csrf
    @method('DELETE')
    ...
</form>
```

Laravel membaca hidden input tersebut dan meneruskan request ke method Controller yang sesuai.

10.2.5 Laravel Storage untuk Upload File

Laravel Storage adalah sistem pengelolaan file bawaan Laravel yang menyediakan antarmuka terpadu untuk menyimpan, mengambil, dan menghapus file. Seluruh operasi file dilakukan melalui satu antarmuka yang sama tanpa perlu mempedulikan di mana file sebenarnya disimpan. Aplikasi yang awalnya menyimpan file di server lokal dapat dipindahkan ke layanan penyimpanan cloud hanya dengan mengubah konfigurasi tanpa menyentuh kode Controller sama sekali.

Pada pendekatan yang telah digunakan di bab-bab sebelumnya, file disimpan langsung ke folder `uploads/` menggunakan fungsi `move_uploaded_file()`. Laravel Storage menawarkan pendekatan yang lebih terstruktur melalui method-method yang sudah tersedia.

File yang diupload melalui form HTML tersedia di Controller melalui objek `$request`. Berikut cara menyimpan file menggunakan Laravel Storage:

```
if ($request->hasFile('gambar')) {
    $file = $request->file('gambar');
    $namaFile = time() . '_' . $file->getClientOriginalName();
```

```
}
    $file->storeAs('public/gambar', $namaFile);
}
```

Penjelasan kode di atas:

- `$request->hasFile('gambar')` memeriksa apakah file dengan nama field `gambar` sudah diunggah.
- `$request->file('gambar')` mengambil file yang diunggah sebagai objek `UploadedFile`.
- `time() . '_' . $file->getClientOriginalName()` menghasilkan nama file unik dengan menambahkan timestamp di depan nama file asli untuk menghindari konflik nama file.
- `$file->storeAs('public/gambar', $namaFile)` menyimpan file ke folder `storage/app/public/gambar/`. Folder `public` di sini merujuk ke disk `public` yang sudah dikonfigurasi di Laravel, bukan folder `public/` di root proyek.

Agar file yang tersimpan di `storage/app/public/` dapat diakses melalui browser, perlu dibuat symbolic link dari `public/storage` ke `storage/app/public/`. Perintah ini sudah dijalankan pada Bab 9:

```
php artisan storage:link
```

Setelah symbolic link dibuat, file dapat diakses melalui browser menggunakan helper `asset()`:

```
asset('storage/gambar/' . $namaFile)
```

Untuk menghapus file dari storage saat data dihapus dari database, gunakan Facade `Storage`:

```
use Illuminate\Support\Facades\Storage;
Storage::delete('public/gambar/' . $namaFile);
```

Pada bab ini digunakan dua folder storage yang terpisah untuk setiap entitas:

```
storage/app/public/
├── foto/           ← foto profil penulis
└── gambar/        ← gambar artikel
```

Pemisahan folder ini memudahkan pengelolaan file dan menghindari konflik nama file antar entitas yang berbeda.

10.2.6 Validasi Input di Laravel

Validasi input adalah proses memeriksa apakah data yang dikirim pengguna melalui form sudah sesuai dengan format dan aturan yang diharapkan sebelum data tersebut diproses lebih lanjut. Laravel menyediakan sistem validasi bawaan yang lengkap dan ekspresif sehingga tidak perlu menulis logika validasi secara manual seperti pada pendekatan PHP prosedural.

Pada pendekatan yang telah digunakan di bab-bab sebelumnya, validasi ditulis secara manual menggunakan kondisi `if` dan fungsi PHP seperti `empty()` dan `isset()`. Di Laravel, validasi dilakukan melalui method `validate()` yang tersedia di objek `$request`:

```
$request->validate([
    'judul'      => 'required|string|max:255',
    'isi'        => 'required|string',
    'id_penulis' => 'required|exists:penulis,id',
    'id_kategori' => 'required|exists:kategori_artikel,id',
    'gambar'    => 'required|image|mimes:jpg,jpeg,png|max:2048',
]);
```

```
] );
```

Setiap aturan validasi didefinisikan sebagai string dengan beberapa aturan yang dipisahkan oleh karakter `|`. Berikut penjelasan aturan validasi yang digunakan pada bab ini:

- required** memastikan field tidak boleh kosong.
- string** memastikan nilai field berupa teks.
- max:255** memastikan panjang teks tidak melebihi 255 karakter.
- exists:penulis,id** memastikan nilai field ada di kolom `id` tabel `penulis`. Aturan ini digunakan untuk memvalidasi foreign key.
- image** memastikan file yang diunggah adalah gambar.
- mimes:jpg,jpeg,png** memastikan tipe file yang diunggah adalah JPG atau PNG.
- max:2048** memastikan ukuran file tidak melebihi 2048 kilobyte atau 2 megabyte.
- nullable** memperbolehkan field bernilai kosong atau null. Digunakan untuk field yang tidak wajib diisi.
- unique:penulis,user_name** memastikan nilai field belum ada di kolom `user_name` tabel `penulis`. Digunakan untuk memvalidasi keunikan data seperti username.

Jika validasi gagal, Laravel secara otomatis mengarahkan pengguna kembali ke halaman sebelumnya beserta pesan error dan nilai input yang sudah diisi. Pesan error ditampilkan di View menggunakan directive Blade `@error`:

```
<input type="text" name="judul" value="{{ old('judul') }}"
      class="form-control @error('judul') is-invalid @enderror">

@error('judul')
    <div class="invalid-feedback">{{ $message }}</div>
@enderror
```

Penjelasan kode di atas:

- `@error('judul')` memeriksa apakah ada pesan error untuk field `judul`. Jika ada, blok di dalamnya akan ditampilkan.
- `{{ $message }}` menampilkan pesan error yang dihasilkan oleh Laravel untuk field tersebut.
- `is-invalid` adalah kelas Bootstrap yang mengubah tampilan input menjadi merah sebagai indikasi error.
- `old('judul')` mengisi kembali nilai input dengan nilai yang sebelumnya dimasukkan pengguna sehingga pengguna tidak perlu mengisi ulang seluruh form dari awal.

10.2.7 PRG Pattern dan Session Flash Message

PRG adalah singkatan dari Post/Redirect/Get. PRG adalah pola desain yang digunakan untuk mencegah pengiriman form berulang jika pengguna menekan tombol refresh setelah melakukan submit. Tanpa pola ini, menekan tombol refresh setelah submit akan mengirimkan ulang data yang sama ke server sehingga data bisa tersimpan lebih dari satu kali.

Berikut ilustrasi alur PRG pada operasi tambah data:

- Pengguna mengisi form dan menekan tombol Simpan
- Browser mengirim POST request ke server
- Controller memproses data dan menyimpannya ke database
- Controller mengirim redirect (GET request) ke halaman daftar data
- Browser mengikuti redirect dan menampilkan halaman daftar data
- Jika pengguna menekan refresh, yang diulang adalah GET request ke halaman daftar, bukan POST request pengiriman form

Di Laravel, redirect setelah operasi berhasil dilakukan menggunakan helper `redirect()`:

```
return redirect()->route('artikel.index');
```

Untuk menyertakan pesan notifikasi hasil operasi, digunakan method `with()` yang menyimpan pesan ke session flash. Session flash adalah data session yang hanya tersedia untuk satu request berikutnya dan otomatis dihapus setelahnya:

```
return redirect()->route('artikel.index')
->with('sukses', 'Artikel berhasil disimpan.');
```

Pesan flash ditampilkan di View menggunakan directive Blade:

```
@if(session('sukses'))
    <div class="alert alert-success alert-dismissible fade show" role="alert">
        {{ session('sukses') }}
        <button type="button" class="btn-close" data-bs-
dismiss="alert"></button>
    </div>
@endif
```

Pada bab ini digunakan dua jenis pesan flash untuk membedakan hasil operasi yang berhasil dan yang gagal:

- sukses** digunakan untuk memberitahu pengguna bahwa operasi berhasil dilakukan, misalnya data berhasil disimpan, diperbarui, atau dihapus.
- gagal** digunakan untuk memberitahu pengguna bahwa operasi gagal dilakukan, misalnya data tidak ditemukan atau terjadi kesalahan saat menghapus file.

Berikut contoh penggunaan kedua jenis pesan flash di Controller:

```
// Jika operasi berhasil
return redirect()->route('artikel.index')
->with('sukses', 'Artikel berhasil disimpan.');
```

```
// Jika operasi gagal
return redirect()->route('artikel.index')
->with('gagal', 'Artikel gagal dihapus.');
```

Pesan flash hanya perlu didefinisikan sekali di layout utama Blade sehingga tersedia di seluruh halaman tanpa perlu menulis ulang kode yang sama di setiap View.

10.3 Tahap-tahap Praktikum

Pada bagian ini, proyek `aplikasi-blog` yang telah dibangun pada Bab 9 dikembangkan menjadi sistem manajemen konten yang lengkap. Pengembangan dimulai dengan menyiapkan Model, Eloquent Relationship, Resource Controller, dan Route, dilanjutkan dengan membangun layout utama yang diwarisi oleh seluruh View. Dashboard diperbarui menjadi halaman penyambut yang terintegrasi dengan layout baru. Setelah persiapan selesai, operasi CRUD diimplementasikan secara bertahap untuk tiga entitas yaitu kategori artikel, penulis, dan artikel.

Persiapan

Langkah 1. Membuka kembali proyek `aplikasi-blog`

Buka Command Prompt atau PowerShell, lalu pindah ke folder proyek:

```
cd C:\xampp\htdocs\aplikasi-blog
```

Buka proyek di VS Code:

```
code .
```

Pastikan MySQL sudah berjalan melalui XAMPP Control Panel. Selanjutnya jalankan development server:

```
php artisan serve
```

Buka browser dan akses alamat berikut untuk memastikan proyek berjalan dengan benar:

```
http://localhost:8000
```

Browser akan menampilkan halaman login seperti yang telah dibangun pada Bab 9. Jika halaman login tampil dengan benar, proyek siap dikembangkan lebih lanjut.

Langkah 2. Membuat Model `Artikel`, `Penulis`, dan `KategoriArtikel`

Ketiga Model dibuat sekaligus menggunakan perintah Artisan. Buka terminal baru di VS Code dengan menekan `Ctrl+```, pastikan terminal berada di folder proyek `aplikasi-blog`, lalu jalankan perintah berikut satu per satu:

```
php artisan make:model Artikel
php artisan make:model Penulis
php artisan make:model KategoriArtikel
```

Jika berhasil, terminal akan menampilkan pesan seperti berikut untuk setiap perintah:

```
INFO  Model [app/Models/Artikel.php] created successfully.
INFO  Model [app/Models/Penulis.php] created successfully.
INFO  Model [app/Models/KategoriArtikel.php] created successfully.
```

Laravel secara otomatis membuat ketiga file Model di folder `app/Models/`. Selanjutnya setiap Model perlu disesuaikan agar merujuk ke tabel yang benar dan mendefinisikan kolom yang dapat diisi secara massal.

Model `Artikel`

Buka file `app/Models/Artikel.php` dan ubah seluruh isinya menjadi seperti berikut:

```
<?php
```

```

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Artikel extends Model
{
    protected $table = 'artikel';

    public $timestamps = false;

    protected $fillable = [
        'id_penulis',
        'id_kategori',
        'judul',
        'isi',
        'gambar',
        'hari_tanggal',
    ];

    public function penulis()
    {
        return $this->belongsTo(Penulis::class, 'id_penulis');
    }

    public function kategori()
    {
        return $this->belongsTo(KategoriArtikel::class, 'id_kategori');
    }
}

```

Model Penulis

Buka file `app/Models/Penulis.php` dan ubah seluruh isinya menjadi seperti berikut:

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Penulis extends Model
{
    protected $table = 'penulis';

    public $timestamps = false;

    protected $fillable = [
        'nama_depan',
        'nama_belakang',
        'user_name',
        'password',
        'foto',
    ];

    protected $hidden = [
        'password',
    ];

    public function artikel()
    {
        return $this->hasMany(Artikel::class, 'id_penulis');
    }
}

```

Model KategoriArtikel

Buka file `app/Models/KategoriArtikel.php` dan ubah seluruh isinya menjadi seperti berikut:

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class KategoriArtikel extends Model
{
    protected $table = 'kategori_artikel';

    public $timestamps = false;

    protected $fillable = [
        'nama_kategori',
        'keterangan',
    ];

    public function artikel()
    {
        return $this->hasMany(Artikel::class, 'id_kategori');
    }
}
```

Perlu diperhatikan beberapa hal terkait ketiga Model di atas:

- Properti `$table` didefinisikan secara eksplisit di setiap Model karena secara default Laravel mengasumsikan nama tabel adalah bentuk jamak dari nama Model. Tanpa properti ini, Laravel akan mencari tabel `artikels`, `penulises`, dan `kategori_artikels` yang tidak ada di database.
- Properti `$timestamps = false` menonaktifkan fitur timestamps otomatis Laravel. Secara default, Eloquent mengasumsikan setiap tabel memiliki kolom `created_at` dan `updated_at` dan akan mencoba mengisi kedua kolom tersebut secara otomatis saat data disimpan atau diperbarui. Karena ketiga tabel yang digunakan tidak memiliki kolom tersebut, properti ini wajib didefinisikan agar tidak terjadi error.
- Properti `$fillable` mendefinisikan kolom yang dapat diisi secara massal. Kolom yang tidak terdaftar di `$fillable` tidak dapat diisi secara massal sehingga aplikasi terlindungi dari serangan mass assignment.
- Properti `$hidden` pada Model `Penulis` memastikan kolom `password` tidak ikut ditampilkan saat data Model dikonversi ke format JSON atau array.

Simpan ketiga file tersebut dengan menekan `Ctrl+S`.

Langkah 3. Mendefinisikan Eloquent Relationship

Setelah ketiga Model dibuat, langkah selanjutnya adalah mendefinisikan relasi antar Model menggunakan Eloquent Relationship. Relasi didefinisikan langsung di dalam file Model masing-masing.

Model Artikel

Buka kembali file `app/Models/Artikel.php` dan tambahkan dua method relationship berikut:

```
<?php
```

```

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Artikel extends Model
{
    protected $table = 'artikel';

    protected $fillable = [
        'id_penulis',
        'id_kategori',
        'judul',
        'isi',
        'gambar',
        'hari_tanggal',
    ];

    public function penulis()
    {
        return $this->belongsTo(Penulis::class, 'id_penulis');
    }

    public function kategori()
    {
        return $this->belongsTo(KategoriArtikel::class, 'id_kategori');
    }
}

```

Method `penulis()` mendefinisikan relasi bahwa setiap artikel dimiliki oleh satu penulis melalui kolom `id_penulis`. Method `kategori()` mendefinisikan relasi bahwa setiap artikel termasuk dalam satu kategori melalui kolom `id_kategori`.

Model Penulis

Buka kembali file `app/Models/Penulis.php` dan tambahkan satu method relationship berikut:

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Penulis extends Model
{
    protected $table = 'penulis';

    protected $fillable = [
        'nama_depan',
        'nama_belakang',
        'user_name',
        'password',
        'foto',
    ];

    protected $hidden = [
        'password',
    ];

    public function artikel()
    {
        return $this->hasMany(Artikel::class, 'id_penulis');
    }
}

```

```
}
```

Method `artikel()` mendefinisikan relasi bahwa satu penulis dapat memiliki banyak artikel melalui kolom `id_penulis` di tabel `artikel`.

Model `KategoriArtikel`

Buka kembali file `app/Models/KategoriArtikel.php` dan tambahkan satu method relationship berikut:

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class KategoriArtikel extends Model
{
    protected $table = 'kategori_artikel';

    protected $fillable = [
        'nama_kategori',
        'keterangan',
    ];

    public function artikel()
    {
        return $this->hasMany(Artikel::class, 'id_kategori');
    }
}
```

Method `artikel()` mendefinisikan relasi bahwa satu kategori dapat memiliki banyak artikel melalui kolom `id_kategori` di tabel `artikel`.

Simpan ketiga file tersebut dengan menekan `Ctrl+S`.

Untuk memverifikasi bahwa relationship sudah didefinisikan dengan benar, jalankan perintah berikut di terminal untuk membuka Laravel Tinker:

```
php artisan tinker
```

Jalankan perintah berikut di dalam Tinker untuk mencoba mengambil satu artikel beserta data penulis dan kategorinya:

```
App\Models\Artikel::with('penulis', 'kategori')->first();
```

Jika relationship sudah benar, Tinker akan menampilkan data artikel lengkap beserta data penulis dan kategori yang berelasi. Ketik `exit` untuk keluar dari Tinker.

Langkah 4. Membuat tiga Resource Controller

Ketiga Resource Controller dibuat menggunakan perintah Artisan dengan menambahkan flag `--resource`. Jalankan perintah berikut satu per satu di terminal:

```
php artisan make:controller ArtikelController --resource
php artisan make:controller PenulisController --resource
php artisan make:controller KategoriArtikelController --resource
```

Jika berhasil, terminal akan menampilkan pesan seperti berikut untuk setiap perintah:

```
INFO Controller [app/Http/Controllers/ArtikelController.php] created
successfully.
INFO Controller [app/Http/Controllers/PenulisController.php] created
successfully.
INFO Controller [app/Http/Controllers/KategoriArtikelController.php] created
successfully.
```

Buka salah satu file Controller yang baru dibuat, misalnya `app/Http/Controllers/ArtikelController.php`. Laravel sudah menghasilkan ketujuh method standar Resource Controller dalam keadaan kosong:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class ArtikelController extends Controller
{
    public function index()
    {
        //
    }

    public function create()
    {
        //
    }

    public function store(Request $request)
    {
        //
    }

    public function show(string $id)
    {
        //
    }

    public function edit(string $id)
    {
        //
    }

    public function update(Request $request, string $id)
    {
        //
    }

    public function destroy(string $id)
    {
        //
    }
}
```

Ketiga Controller ini akan diisi dengan logika bisnis secara bertahap pada langkah-langkah berikutnya. Untuk saat ini, biarkan seluruh method dalam keadaan kosong.

Langkah 5. Mendefinisikan Route resource

Buka file `routes/web.php` di VS Code. File ini sudah berisi Route login, logout, dan dashboard dari Bab 9. Tambahkan Route resource untuk ketiga Controller yang baru dibuat dengan menambahkan baris berikut:

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\LoginController;
use App\Http\Controllers\DashboardController;
use App\Http\Controllers\ArtikelController;
use App\Http\Controllers\PenulisController;
use App\Http\Controllers\KategoriArtikelController;

// Route untuk halaman login
Route::get('/login', [LoginController::class, 'index'])->name('login')->middleware('guest');
Route::post('/login', [LoginController::class, 'proses'])->name('login.proses')->middleware('guest');

// Route untuk logout
Route::post('/logout', [LoginController::class, 'logout'])->name('logout')->middleware('auth');

// Route yang dilindungi middleware auth
Route::middleware('auth')->group(function () {

    // Route untuk halaman dashboard
    Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard');

    // Route resource untuk ketiga entitas
    Route::resource('artikel', ArtikelController::class)->except(['show']);
    Route::resource('penulis', PenulisController::class)->except(['show']);
    Route::resource('kategori', KategoriArtikelController::class)->except(['show']);

});

// Redirect halaman utama ke login
Route::get('/', function () {
    return redirect()->route('login');
});
```

Penjelasan perubahan yang dilakukan:

- Route group dengan middleware auth** Seluruh Route dashboard dan CRUD dikelompokkan dalam satu Route group dengan middleware `auth`. Pendekatan ini lebih ringkas dibandingkan menambahkan `->middleware('auth')` satu per satu di setiap Route.
- `->except(['show'])`** Method `show` tidak diimplementasikan pada bab ini karena aplikasi tidak memerlukan halaman detail untuk setiap entitas. Method `except()` menghapus Route `show` dari daftar Route yang didaftarkan sehingga hanya enam Route yang aktif untuk setiap entitas.

Simpan file `routes/web.php` dengan menekan `Ctrl+S`.

Verifikasi seluruh Route yang terdaftar dengan menjalankan perintah berikut di terminal:

```
php artisan route:list
```


Terminal akan menampilkan daftar seluruh Route yang aktif. Pastikan Route untuk artikel, penulis, dan kategori sudah muncul dalam daftar tersebut.

Langkah 6. Menyiapkan folder storage untuk gambar

Pada Bab 9, symbolic link dari `public/storage` ke `storage/app/public/` sudah dibuat menggunakan perintah `php artisan storage:link`. Folder `foto/` untuk menyimpan foto profil penulis juga sudah tersedia. Pada langkah ini, folder `gambar/` untuk menyimpan gambar artikel perlu ditambahkan.

Buat folder `gambar` di dalam `storage/app/public/` melalui VS Code dengan cara klik kanan pada folder `public` di dalam `storage/app/`, pilih **New Folder**, lalu ketik `gambar` dan tekan Enter.

Setelah folder dibuat, struktur folder storage menjadi seperti berikut:

```
storage/app/public/  
├── foto/           ← foto profil penulis (sudah ada dari Bab 9)  
└── gambar/        ← gambar artikel (baru dibuat)
```

Untuk memastikan folder `gambar/` dapat diakses melalui browser, buat file `.gitkeep` kosong di dalamnya. File ini berfungsi sebagai placeholder agar folder tidak dianggap kosong oleh sistem:

Buka terminal dan jalankan perintah berikut:

```
echo "" > storage/app/public/gambar/.gitkeep
```

Verifikasi bahwa folder sudah dapat diakses melalui browser dengan membuka alamat berikut:

```
http://localhost:8000/storage/gambar/
```

Jika browser menampilkan halaman kosong atau pesan direktori tidak dapat diakses, folder sudah terhubung dengan benar melalui symbolic link yang dibuat pada Bab 9.

Langkah 7. Membuat layout utama `layouts/app.blade.php`

Layout utama adalah file Blade yang berisi struktur HTML yang digunakan bersama oleh seluruh View dalam aplikasi. Dengan layout utama, header, sidebar, dan struktur konten cukup ditulis sekali dan diwarisi oleh seluruh View menggunakan directive `@extends` dan `@section`.

Buat folder `layouts` di dalam `resources/views/` melalui VS Code dengan cara klik kanan pada folder `views` di panel Explorer, pilih **New Folder**, lalu ketik `layouts` dan tekan Enter.

Selanjutnya buat file baru bernama `app.blade.php` di dalam folder `layouts`. Isi file tersebut dengan kode berikut:

```
<!DOCTYPE html>  
<html lang="id">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>@yield('title', 'Sistem Manajemen Blog')</title>  
    <link  
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"  
rel="stylesheet">  
    <style>  
        body {  
            background-color: #f4f4f9;
```

```
        font-size: 14px;
    }
    .header {
        background-color: #2C3E50;
        color: #ffffff;
        padding: 12px 20px;
    }
    .header-title {
        font-size: 15px;
        font-weight: 500;
        margin: 0;
    }
    .header-sub {
        font-size: 11px;
        color: #aaaaaa;
        margin: 0;
    }
    .sidebar {
        width: 210px;
        min-height: calc(100vh - 52px);
        background-color: #ffffff;
        border-right: 1px solid #f0f0f0;
        padding: 16px 0;
        flex-shrink: 0;
    }
    .avatar-area {
        padding: 0 14px 16px;
        border-bottom: 1px solid #f0f0f0;
        margin-bottom: 12px;
    }
    .avatar-circle {
        width: 48px;
        height: 48px;
        border-radius: 50%;
        object-fit: cover;
        border: 2px solid #e9ecef;
        margin-bottom: 8px;
    }
    .avatar-greeting {
        font-size: 11px;
        color: #868e96;
        margin: 0;
    }
    .avatar-name {
        font-size: 13px;
        font-weight: 500;
        color: #212529;
        margin: 0;
    }
    .sidebar-label {
        font-size: 10px;
        color: #adb5bd;
        padding: 0 14px 8px;
        text-transform: uppercase;
        letter-spacing: 0.06em;
        font-weight: 600;
    }
    .nav-item {
        padding: 9px 14px;
        font-size: 13px;
        color: #555555;
        border-left: 2px solid transparent;
        cursor: pointer;
        display: flex;
```

```

        align-items: center;
        gap: 8px;
        text-decoration: none;
    }
    .nav-item:hover {
        background-color: #f4f4f9;
        color: #333333;
    }
    .nav-item.active {
        background-color: #e8f5e9;
        color: #2e7d32;
        border-left-color: #4CAF50;
        font-weight: 600;
    }
    .sidebar-bottom {
        padding: 12px 14px 0;
        border-top: 1px solid #f0f0f0;
        margin-top: auto;
    }
    .main-content {
        flex: 1;
        padding: 24px;
    }
}
</style>
</head>
<body>

    <div class="header d-flex align-items-center">
        <div>
            <p class="header-title">Sistem Manajemen Blog (CMS)</p>
            <p class="header-sub">db_blog</p>
        </div>
    </div>

    <div class="d-flex">
        <div class="sidebar d-flex flex-column">
            <div class="avatar-area">
                
                <p class="avatar-greeting">Halo,</p>
                <p class="avatar-name">
                    {{ Auth::user()->nama_depan }}
                    {{ Auth::user()->nama_belakang }}
                </p>
            </div>

            <div class="sidebar-label">Menu Utama</div>

            <a href="{{ route('dashboard') }}"
                class="nav-item {{ request()->routeIs('dashboard') ? 'active' :
'' }}">
                Dashboard
            </a>
            <a href="{{ route('artikel.index') }}"
                class="nav-item {{ request()->routeIs('artikel.*') ? 'active' :
'' }}">
                Kelola Artikel
            </a>
            <a href="{{ route('penulis.index') }}"
                class="nav-item {{ request()->routeIs('penulis.*') ? 'active' :
'' }}">
                Kelola Penulis
            </a>

```

```

<a href="{{ route('kategori.index') }}"
    class="nav-item {{ request()->routeIs('kategori.*') ? 'active'
: '' }}">
    Kelola Kategori
</a>

<div class="sidebar-bottom mt-auto">
    <form action="{{ route('logout') }}" method="POST">
        @csrf
        <button type="submit"
            class="btn btn-sm w-100"
            style="background-color:#fff0f0;          color:#c0392b;
border: 1px solid #f5c6c6;">
            Keluar
        </button>
    </form>
</div>
</div>

<div class="main-content">

    @if(session('sukses'))
        <div class="alert alert-success alert-dismissible fade show"
role="alert">
            {{ session('sukses') }}
            <button type="button" class="btn-close" data-bs-
dismiss="alert"></button>
        </div>
    @endif

    @if(session('gagal'))
        <div class="alert alert-danger alert-dismissible fade show"
role="alert">
            {{ session('gagal') }}
            <button type="button" class="btn-close" data-bs-
dismiss="alert"></button>
        </div>
    @endif

    @yield('content')

</div>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min
.js"></script>
</body>
</html>

```

Penjelasan bagian-bagian penting pada layout di atas:

- a. **@yield('title', 'Sistem Manajemen Blog')** Directive ini menyediakan slot untuk judul halaman. Setiap View yang mewarisi layout ini dapat mendefinisikan judul halamannya sendiri menggunakan **@section('title', 'Judul Halaman')**. Jika tidak didefinisikan, judul default Sistem Manajemen Blog akan digunakan.
- b. **Auth::user()** Mengambil data pengguna yang sedang login langsung di layout sehingga foto profil dan nama pengguna tersedia di seluruh halaman tanpa perlu dikirim dari setiap Controller.

- c. `request()->routeIs('artikel.*')` Memeriksa apakah Route yang sedang aktif termasuk dalam kelompok Route `artikel`. Jika ya, class `active` ditambahkan pada item menu yang sesuai sehingga menu yang sedang aktif terlihat berbeda secara visual.
- d. `@yield('content')` Directive ini menyediakan slot utama untuk konten halaman. Setiap View mendefinisikan kontennya di dalam `@section('content') ... @endsection`.
- e. **Session flash message** Pesan sukses dan gagal ditampilkan di layout utama sehingga tersedia di seluruh halaman tanpa perlu menulis ulang kode yang sama di setiap View.

Simpan file tersebut dengan menekan Ctrl+S.

Dashboard

Langkah 8. Memperbarui View dashboard

Buka file `resources/views/dashboard/index.blade.php` yang sudah dibuat pada Bab 9. Hapus seluruh isinya dan ganti dengan kode berikut:

```
@extends('layouts.app')

@section('title', 'Dashboard')

@section('content')

<div class="d-flex justify-content-center align-items-center"
    style="min-height: calc(100vh - 160px);">
    <div class="card border-0 shadow-sm" style="max-width: 480px; width: 100%;
border-radius: 12px;">
        <div class="card-body p-4 p-md-5 text-center">

            <div class="mb-4">
                <div style="width: 64px; height: 64px; border-radius: 50%;
background-color: #e8f5e9; display: flex;
align-items: center; justify-content: center;
margin: 0 auto 16px;">
                    <svg xmlns="http://www.w3.org/2000/svg" width="28"
height="28"
fill="none" viewBox="0 0 24 24" stroke="#2e7d32"
stroke-width="1.5">
                        <path stroke-linecap="round" stroke-linejoin="round"
d="M3 12l2-2m0 0l7-7 7 7M5 10v10a1 1 0 01 1h3m10-
11l2
                2m-2-2v10a1 1 0 01 1 1h-3m-6 0a1 1 0 001-1v-4a1 1
0
                011-1h2a1 1 0 011 1v4a1 1 0 001 1m-6 0h6" />
                    </svg>
                </div>
                <h5 class="fw-semibold mb-1">
                    Selamat datang,
                    <span style="color: #2e7d32;">{{ $nama_lengkap }}</span>
                </h5>
                <p class="text-muted small mb-0">
                    Silakan pilih menu di sebelah kiri untuk mulai mengelola
konten blog.
                </p>
            </div>

            <hr class="my-3">

            <div class="row g-3 text-start">
```

```

        <div class="col-6">
            <div class="p-3 rounded"
                style="background-color: #f8f9fa;">
                <div class="text-uppercase fw-semibold mb-1"
                    style="font-size: 10px; color: #adb5bd;
                    letter-spacing: 0.05em;">
                    Login sebagai
                </div>
                <div style="font-size: 12px; font-weight: 500;
                    color: #212529;">
                    {{ $nama_lengkap }}
                </div>
            </div>
        </div>
        <div class="col-6">
            <div class="p-3 rounded"
                style="background-color: #f8f9fa;">
                <div class="text-uppercase fw-semibold mb-1"
                    style="font-size: 10px; color: #adb5bd;
                    letter-spacing: 0.05em;">
                    Waktu login
                </div>
                <div style="font-size: 12px; font-weight: 500;
                    color: #212529;">
                    {{ $waktu_login }}
                </div>
            </div>
        </div>
    </div>
</div>
</div>
@endsection

```

Penjelasan bagian-bagian penting pada View di atas:

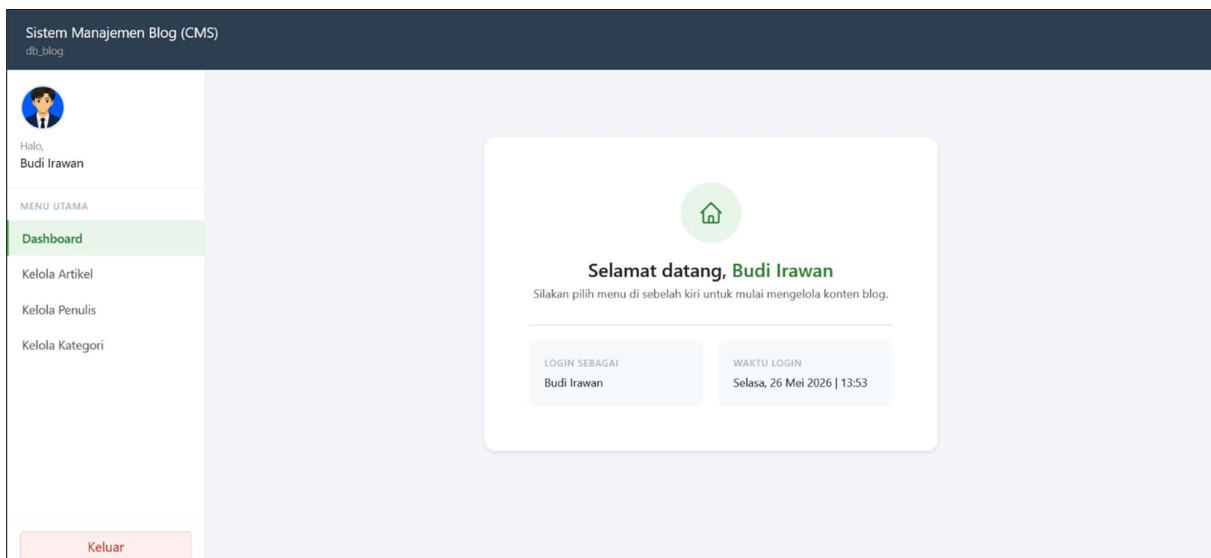
- @extends('layouts.app')** Directive ini memberitahu Laravel bahwa View ini mewarisi layout utama yang sudah dibuat pada Langkah 7. Seluruh struktur HTML header, sidebar, dan session flash message dari layout utama akan digunakan secara otomatis.
- @section('title', 'Dashboard')** Mendefinisikan judul halaman yang akan ditampilkan di tab browser. Nilai ini mengisi slot @yield('title') di layout utama.
- @section('content') ... @endsection** Mendefinisikan konten utama halaman yang akan mengisi slot @yield('content') di layout utama.
- {{ \$nama_lengkap }}** dan **{{ \$waktu_login }}** Variabel ini dikirim dari DashboardController yang sudah dibuat pada Bab 9 sehingga tidak perlu ada perubahan pada Controller.

Simpan file tersebut dengan menekan Ctrl+S.

Buka browser dan akses alamat berikut setelah login:

```
http://localhost:8000/dashboard
```

browser akan menampilkan halaman dashboard yang sudah diperbarui dengan layout sidebar di sebelah kiri dan kartu selamat datang di konten utama seperti pada Gambar 10.1.



Gambar 10.1 Halaman dashboard setelah diperbarui dengan layout utama

CRUD Kategori Artikel

Langkah 9. Mengimplementasikan index kategori

Langkah ini mengimplementasikan halaman daftar kategori artikel. Halaman ini menampilkan seluruh data kategori dalam bentuk tabel beserta tombol tambah, edit, dan hapus.

Mengisi method `index` di `KategoriArtikelController`

Buka file `app/Http/Controllers/KategoriArtikelController.php` dan tambahkan import Model serta isi method `index` seperti berikut:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\KategoriArtikel;

class KategoriArtikelController extends Controller
{
    public function index()
    {
        $kategori = KategoriArtikel::orderBy('nama_kategori', 'asc')->get();

        return view('kategori.index', compact('kategori'));
    }

    public function create()
    {
        //
    }

    public function store(Request $request)
    {
        //
    }

    public function edit(string $id)
    {
        //
    }
}
```

```

    }

    public function update(Request $request, string $id)
    {
        //
    }

    public function destroy(string $id)
    {
        //
    }
}

```

Membuat View index kategori

Buat folder kategori di dalam resources/views/ melalui VS Code. Selanjutnya buat file baru bernama index.blade.php di dalam folder kategori. Isi file tersebut dengan kode berikut:

```

@extends('layouts.app')

@section('title', 'Kelola Kategori Artikel')

@section('content')

<div class="d-flex justify-content-between align-items-center mb-3">
    <h6 class="fw-semibold mb-0" style="color: #333333;">Data Kategori
    Artikel</h6>
    <a href="{{ route('kategori.create') }}" class="btn btn-sm btn-success">
        + Tambah Kategori
    </a>
</div>

<div class="card border-0 shadow-sm">
    <div class="card-body p-0">
        <table class="table table-hover mb-0">
            <thead>
                <tr style="background-color: #fafafa;">
                    <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                        text-transform: uppercase; letter-spacing: 0.05em;">
                        No
                    </th>
                    <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                        text-transform: uppercase; letter-spacing: 0.05em;">
                        Nama Kategori
                    </th>
                    <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                        text-transform: uppercase; letter-spacing: 0.05em;">
                        Keterangan
                    </th>
                    <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                        text-transform: uppercase; letter-spacing: 0.05em;">
                        Aksi
                    </th>
                </tr>
            </thead>
            <tbody>
                @foreach($kategori as $index => $item)
                    <tr>
                        <td class="px-3 py-2" style="font-size: 13px;">
                            {{ $index + 1 }}

```



```

</td>
<td class="px-3 py-2" style="font-size: 13px;">
    {{ $item->nama_kategori }}
</td>
<td class="px-3 py-2" style="font-size: 13px;
    color: #666666;">
    {{ $item->keterangan ?? '-' }}
</td>
<td class="px-3 py-2">
    <div class="d-flex gap-2">
        <a href="{{ route('kategori.edit', $item->id)
            class="btn btn-sm"
            style="background-color: #e3f2fd;
            color: #1565c0; font-size: 12px;">
            Edit
        </a>
        <form action="{{ route('kategori.destroy',
$items->id) }}"
            method="POST"
            onsubmit="return confirm('Hapus kategori
            @csrf
            @method('DELETE')
            <button type="submit" class="btn btn-sm"
            style="background-color: #ffebee;
            color: #c62828; font-size: 12px;">
            Hapus
            </button>
        </form>
    </div>
</td>
</tr>
@empty
<tr>
<td colspan="4" class="px-3 py-4 text-center"
    style="font-size: 13px; color: #999999;
    font-style: italic;">
    Belum ada data kategori.
</td>
</tr>
@endforelse
</tbody>
</table>
</div>
</div>
@endsection

```

Penjelasan bagian-bagian penting pada View di atas:

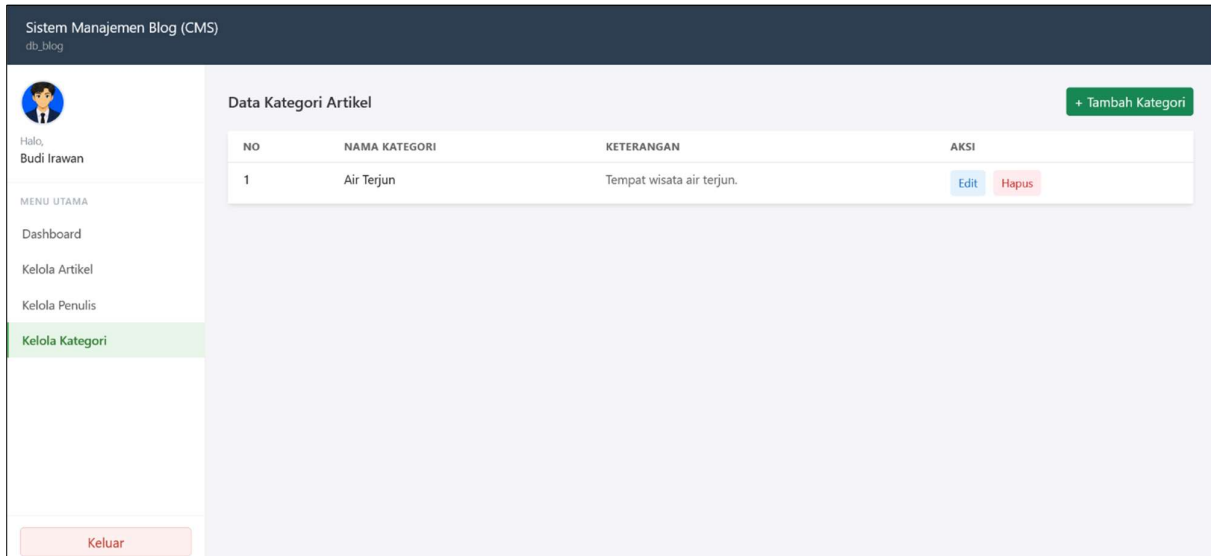
- @forelse(\$kategori as \$index => \$item) ... @empty ... @endforelse** Directive @forelse bekerja seperti foreach namun memiliki blok @empty yang ditampilkan jika koleksi data kosong. Mekanisme ini lebih efektif dibandingkan menggunakan @if dan @foreach secara terpisah.
- {{ \$item->keterangan ?? '-' }}** Operator ?? menampilkan tanda '-' jika kolom keterangan bernilai null.

c. `onsubmit="return confirm('Hapus kategori ini?')"` Menampilkan dialog konfirmasi sebelum form hapus dikirim. Jika pengguna memilih batal, form tidak dikirim.

Simpan file tersebut dengan menekan `Ctrl+S`. Buka browser dan akses:

`http://localhost:8000/kategori`

Browser akan menampilkan halaman daftar kategori seperti pada Gambar 10.2.



Gambar 10.2 Halaman daftar kategori artikel

Langkah 10. Mengimplementasikan tambah kategori

Langkah ini mengimplementasikan fitur tambah kategori artikel. Fitur ini terdiri dari dua bagian yaitu halaman form tambah dan proses penyimpanan data ke database.

Mengisi method `create` dan `store` di `KategoriArtikelController`

Buka kembali file `app/Http/Controllers/KategoriArtikelController.php` dan isi method `create` dan `store` seperti berikut:

```
public function create()
{
    return view('kategori.create');
}

public function store(Request $request)
{
    $request->validate([
        'nama_kategori' => 'required|string|max:100|unique:kategori_artikel,nama_kategori',
        'keterangan' => 'nullable|string',
    ]);

    KategoriArtikel::create([
        'nama_kategori' => $request->nama_kategori,
        'keterangan' => $request->keterangan,
    ]);

    return redirect()->route('kategori.index')
        ->with('sukses', 'Kategori berhasil ditambahkan.');
```

Penjelasan kode di atas:

- Method `create()` hanya mengembalikan View form tambah kategori tanpa perlu mengirim data apapun ke View.
- Method `store()` memvalidasi data yang dikirim dari form, menyimpan data baru ke database menggunakan `KategoriArtikel::create()`, lalu mengarahkan pengguna kembali ke halaman daftar kategori dengan pesan sukses.
- Aturan validasi `unique:kategori_artikel,nama_kategori` memastikan nama kategori belum ada di tabel `kategori_artikel` sehingga tidak ada duplikasi data.

Membuat View form tambah kategori

Buat file baru bernama `create.blade.php` di dalam folder `resources/views/kategori/`. Isi file tersebut dengan kode berikut:

```
@extends('layouts.app')

@section('title', 'Tambah Kategori Artikel')

@section('content')

<div class="d-flex justify-content-between align-items-center mb-3">
    <h6 class="fw-semibold mb-0" style="color: #333333;">Tambah Kategori
    Artikel</h6>
    <a href="{{ route('kategori.index') }}" class="btn btn-sm"
        style="background-color: #f0f0f0; color: #555555;">
        Kembali
    </a>
</div>

<div class="card border-0 shadow-sm">
    <div class="card-body p-4">
        <form action="{{ route('kategori.store') }}" method="POST">
            @csrf

            <div class="mb-3">
                <label for="nama_kategori" class="form-label fw-semibold"
                    style="font-size: 13px;">
                    Nama Kategori <span class="text-danger">*</span>
                </label>
                <input type="text"
                    class="form-control @error('nama_kategori') is-invalid
@enderror"
                    id="nama_kategori"
                    name="nama_kategori"
                    value="{{ old('nama_kategori') }}"
                    placeholder="Masukkan nama kategori">
                @error('nama_kategori')
                <div class="invalid-feedback">{{ $message }}</div>
                @enderror
            </div>

            <div class="mb-4">
                <label for="keterangan" class="form-label fw-semibold"
                    style="font-size: 13px;">
                    Keterangan
                </label>
            </div>
        </form>
    </div>
</div>
```

```

        <textarea class="form-control @error('keterangan') is-invalid
@enderror"
            id="keterangan"
            name="keterangan"
            rows="4"
            placeholder="Masukkan keterangan kategori (opsional)">{{
old('keterangan') }}</textarea>
        @error('keterangan')
        <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>

    <div class="d-flex gap-2 justify-content-end">
        <a href="{{ route('kategori.index') }}" class="btn btn-sm"
            style="background-color: #f0f0f0; color: #555555;">
            Batal
        </a>
        <button type="submit" class="btn btn-sm btn-success">
            Simpan Data
        </button>
    </div>

</form>
</div>
</div>
@endsection

```

Penjelasan bagian-bagian penting pada View di atas:

- *** Tanda bintang merah menandakan field yang wajib diisi.
- @error('nama_kategori') is-invalid @enderror** Menambahkan class `is-invalid` pada input jika validasi untuk field tersebut gagal. Class ini mengubah tampilan border input menjadi merah sebagai indikasi error.
- {{ old('keterangan') }}** Untuk elemen `textarea`, nilai lama ditulis di antara tag pembuka dan penutup, bukan di atribut `value`.

Simpan file tersebut dengan menekan **Ctrl+S**. Buka browser dan akses:

```
http://localhost:8000/kategori/create
```

Browser akan menampilkan halaman form tambah kategori seperti pada Gambar 10.3.

The screenshot shows a web application interface for a CMS. The title bar at the top says 'Sistem Manajemen Blog (CMS)' and 'db_blog'. On the left is a sidebar with a user profile 'Halo, Budi Irawan' and a menu with items: 'Dashboard', 'Kelola Artikel', 'Kelola Penulis', and 'Kelola Kategori' (which is highlighted in green). At the bottom of the sidebar is a 'Keluar' button. The main content area is titled 'Tambah Kategori Artikel' and has a 'Kembali' button in the top right corner. The form itself has two main sections: 'Nama Kategori *' with a text input field containing the placeholder 'Masukkan nama kategori', and 'Keterangan' with a larger text area containing the placeholder 'Masukkan keterangan kategori (opsional)'. At the bottom right of the form are two buttons: 'Batal' and 'Simpan Data'.

Gambar 10.3 Halaman form tambah kategori artikel

Langkah 11. Mengimplementasikan edit kategori

Langkah ini mengimplementasikan fitur edit kategori artikel. Fitur ini terdiri dari dua bagian yaitu halaman form edit yang sudah terisi data lama dan proses pembaruan data ke database.

Mengisi method `edit` dan `update` di `KategoriArtikelController`

Buka kembali file `app/Http/Controllers/KategoriArtikelController.php` dan isi method `edit` dan `update` seperti berikut:

```
public function edit(string $id)
{
    $kategori = KategoriArtikel::findOrFail($id);

    return view('kategori.edit', compact('kategori'));
}

public function update(Request $request, string $id)
{
    $kategori = KategoriArtikel::findOrFail($id);

    $request->validate([
        'nama_kategori' =>
        'required|string|max:100|unique:kategori_artikel,nama_kategori,' . $id,
        'keterangan' => 'nullable|string',
    ]);

    $kategori->update([
        'nama_kategori' => $request->nama_kategori,
        'keterangan' => $request->keterangan,
    ]);

    return redirect()->route('kategori.index')
        ->with('sukses', 'Kategori berhasil diperbarui.');
```

Penjelasan kode di atas:

- Method `edit()` mengambil data kategori berdasarkan `$id` menggunakan `findOrFail()`. Jika data tidak ditemukan, Laravel secara otomatis menghasilkan response 404. Data kategori kemudian dikirim ke View untuk mengisi nilai awal form.
- Method `update()` mengambil data kategori yang sama, memvalidasi data baru, lalu memperbarui data di database menggunakan method `update()` pada instance Model.
- Aturan validasi `unique:kategori_artikel,nama_kategori,' ',$id` memastikan nama kategori unik namun mengecualikan data kategori yang sedang diedit. Tanpa pengecualian ini, validasi akan selalu gagal karena nama kategori yang sama sudah ada di database milik data yang sedang diedit.

Membuat View form edit kategori

Buat file baru bernama `edit.blade.php` di dalam folder `resources/views/kategori/`. Isi file tersebut dengan kode berikut:

```
@extends('layouts.app')

@section('title', 'Edit Kategori Artikel')

@section('content')

<div class="d-flex justify-content-between align-items-center mb-3">
    <h6 class="fw-semibold mb-0" style="color: #333333;">Edit Kategori Artikel</h6>
    <a href="{{ route('kategori.index') }}" class="btn btn-sm" style="background-color: #f0f0f0; color: #555555;">Kembali</a>
</div>

<div class="card border-0 shadow-sm">
    <div class="card-body p-4">
        <form action="{{ route('kategori.update', $kategori->id) }}" method="POST">
            @csrf
            @method('PUT')

            <div class="mb-3">
                <label for="nama_kategori" class="form-label fw-semibold" style="font-size: 13px;">
                    Nama Kategori <span class="text-danger">*</span>
                </label>
                <input type="text" class="form-control @error('nama_kategori') is-invalid @enderror" id="nama_kategori" name="nama_kategori" value="{{ old('nama_kategori', $kategori->nama_kategori) }}" placeholder="Masukkan nama kategori">
                @error('nama_kategori')
                <div class="invalid-feedback">{{ $message }}</div>
                @enderror
            </div>

            <div class="mb-4">
                <label for="keterangan" class="form-label fw-semibold">
```

```

                style="font-size: 13px;">
                Keterangan
            </label>
            <textarea class="form-control @error('keterangan') is-invalid
@enderror"
                id="keterangan"
                name="keterangan"
                rows="4"
                placeholder="Masukkan keterangan kategori (opsional)">{{
old('keterangan', $kategori->keterangan) }}</textarea>
            @error('keterangan')
                <div class="invalid-feedback">{{ $message }}</div>
            @enderror
        </div>

        <div class="d-flex gap-2 justify-content-end">
            <a href="{{ route('kategori.index') }}" class="btn btn-sm"
                style="background-color: #f0f0f0; color: #555555;">
                Batal
            </a>
            <button type="submit" class="btn btn-sm btn-success">
                Simpan Perubahan
            </button>
        </div>

    </form>
</div>
</div>
@endsection

```

Penjelasan bagian penting pada View di atas:

- `@method('PUT')` Directive ini menghasilkan hidden input yang memberitahu Laravel bahwa request ini menggunakan method `PUT` meskipun form HTML hanya mendukung method `POST`.
- `old('nama_kategori', $kategori->nama_kategori)` Argumen kedua pada helper `old()` adalah nilai default yang digunakan jika tidak ada nilai lama dari request sebelumnya. Saat halaman edit pertama kali dibuka, nilai default ini mengisi form dengan data kategori yang sedang diedit.

Simpan file tersebut dengan menekan `Ctrl+S`. Buka browser dan akses halaman daftar kategori, lalu klik tombol **Edit** pada salah satu data. Browser akan menampilkan halaman form edit kategori seperti pada Gambar 10.4.

Gambar 10.4 Halaman form edit kategori artikel

Langkah 12. Mengimplementasikan hapus kategori

Langkah ini mengimplementasikan fitur hapus kategori artikel. Berbeda dengan fitur tambah dan edit yang memerlukan halaman tersendiri, fitur hapus tidak memerlukan View baru karena tombol hapus sudah tersedia di halaman daftar kategori yang dibuat pada Langkah 9.

Mengisi method `destroy` di `KategoriArtikelController`

Buka kembali file `app/Http/Controllers/KategoriArtikelController.php` dan isi method `destroy` seperti berikut:

```
public function destroy(string $id)
{
    $kategori = KategoriArtikel::findOrFail($id);

    try {
        $kategori->delete();

        return redirect()->route('kategori.index')
            ->with('sukses', 'Kategori berhasil dihapus.');
```

```
    } catch (\Exception $e) {
        return redirect()->route('kategori.index')
            ->with('gagal', 'Kategori tidak dapat dihapus karena masih memiliki
artikel.');
```

```
    }
}
```

Penjelasan kode di atas:

- `KategoriArtikel::findOrFail($id)`** Mengambil data kategori berdasarkan `$id`. Jika data tidak ditemukan, Laravel secara otomatis menghasilkan response 404.
- `try ... catch`** Blok `try` mencoba menghapus data kategori. Jika penghapusan berhasil, pengguna diarahkan ke halaman daftar kategori dengan pesan sukses. Blok `catch` menangkap exception yang terjadi jika penghapusan gagal. Penghapusan akan gagal jika kategori masih memiliki artikel yang berelasi karena tabel `artikel` memiliki `constraint ON DELETE RESTRICT`

pada foreign key id_kategori. Dalam kondisi ini, pengguna diarahkan kembali ke halaman daftar kategori dengan pesan gagal yang informatif.

Berikut tampilan keseluruhan file KategoriArtikelController.php setelah seluruh method diisi:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\KategoriArtikel;

class KategoriArtikelController extends Controller
{
    public function index()
    {
        $kategori = KategoriArtikel::orderBy('nama_kategori', 'asc')->get();

        return view('kategori.index', compact('kategori'));
    }

    public function create()
    {
        return view('kategori.create');
    }

    public function store(Request $request)
    {
        $request->validate([
            'nama_kategori' => 'required|string|max:100|unique:kategori_artikel,nama_kategori',
            'keterangan' => 'nullable|string',
        ]);

        KategoriArtikel::create([
            'nama_kategori' => $request->nama_kategori,
            'keterangan' => $request->keterangan,
        ]);

        return redirect()->route('kategori.index')
            ->with('sukses', 'Kategori berhasil ditambahkan.');
```

```

    });

    return redirect()->route('kategori.index')
        ->with('sukses', 'Kategori berhasil diperbarui.');
```

```

    }

    public function destroy(string $id)
    {
        $kategori = KategoriArtikel::findOrFail($id);

        try {
            $kategori->delete();

            return redirect()->route('kategori.index')
                ->with('sukses', 'Kategori berhasil dihapus.');
```

```

        } catch (\Exception $e) {
            return redirect()->route('kategori.index')
                ->with('gagal', 'Kategori tidak dapat dihapus karena masih
memiliki artikel.');
```

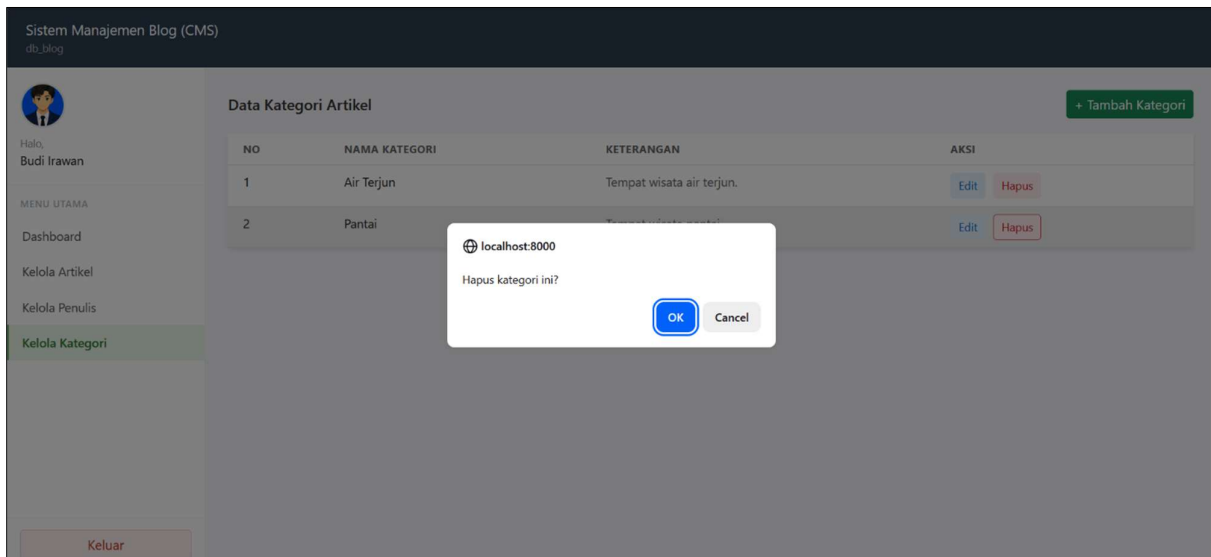
```

        }
    }
}

```

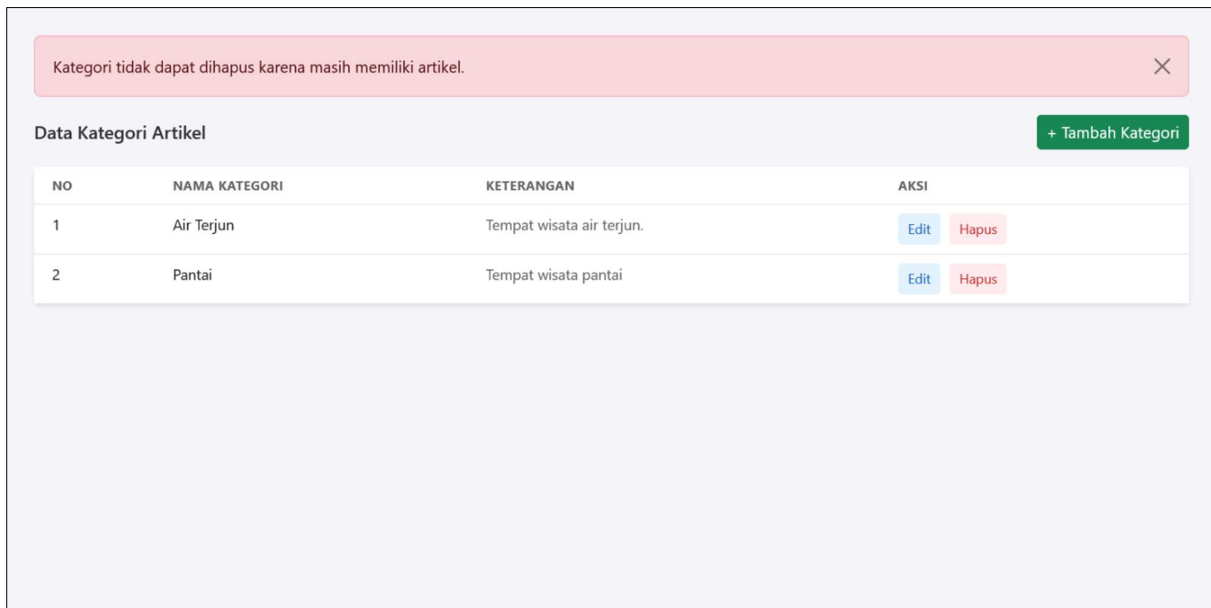
Simpan file tersebut dengan menekan **Ctrl+S**.

Untuk memverifikasi fitur hapus, buka browser dan akses halaman daftar kategori. Klik tombol **Hapus** pada salah satu kategori yang tidak memiliki artikel. Dialog konfirmasi akan muncul seperti pada Gambar 10.5. Klik **OK** untuk melanjutkan penghapusan.



Gambar 10.5 Dialog konfirmasi hapus kategori

Jika kategori yang dihapus masih memiliki artikel, pesan gagal akan ditampilkan di halaman daftar kategori seperti pada Gambar 10.6.



Gambar 10.6 Pesan gagal saat kategori masih memiliki artikel

CRUD Penulis

Langkah 13. Mengimplementasikan index penulis

Langkah ini mengimplementasikan halaman daftar penulis. Halaman ini menampilkan seluruh data penulis dalam bentuk tabel beserta foto profil, tombol tambah, edit, dan hapus.

Mengisi method `index` di `PenulisController`

Buka file `app/Http/Controllers/PenulisController.php` dan tambahkan import Model serta isi method `index` seperti berikut:

```
<?php

namespace App\Http\Controllers;
use Illuminate\Support\Facades\Storage;
use Illuminate\Http\Request;
use App\Models\Penulis;

class PenulisController extends Controller
{
    public function index()
    {
        $penulis = Penulis::orderBy('nama_depan', 'asc')->get();

        return view('penulis.index', compact('penulis'));
    }

    public function create()
    {
        //
    }

    public function store(Request $request)
    {
        //
    }

    public function edit(string $id)
```

```

{
    //

}

public function update(Request $request, string $id)
{
    //
}

public function destroy(string $id)
{
    //
}
}

```

Membuat View index penulis

Buat folder `penulis` di dalam `resources/views/` melalui VS Code. Selanjutnya buat file baru bernama `index.blade.php` di dalam folder `penulis`. Isi file tersebut dengan kode berikut:

```

@extends('layouts.app')

@section('title', 'Kelola Penulis')

@section('content')

<div class="d-flex justify-content-between align-items-center mb-3">
    <h6 class="fw-semibold mb-0" style="color: #333333;">Data Penulis</h6>
    <a href="{{ route('penulis.create') }}" class="btn btn-sm btn-success">
        + Tambah Penulis
    </a>
</div>

<div class="card border-0 shadow-sm">
    <div class="card-body p-0">
        <table class="table table-hover mb-0">
            <thead>
                <tr style="background-color: #fafafa;">
                    <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                        text-transform: uppercase; letter-spacing: 0.05em;">
                        Foto
                    </th>
                    <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                        text-transform: uppercase; letter-spacing: 0.05em;">
                        Nama
                    </th>
                    <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                        text-transform: uppercase; letter-spacing: 0.05em;">
                        Username
                    </th>
                    <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                        text-transform: uppercase; letter-spacing: 0.05em;">
                        Aksi
                    </th>
                </tr>
            </thead>
            <tbody>
                @forelse($penulis as $item)
                    <tr>
                        <td class="px-3 py-2">

```

```

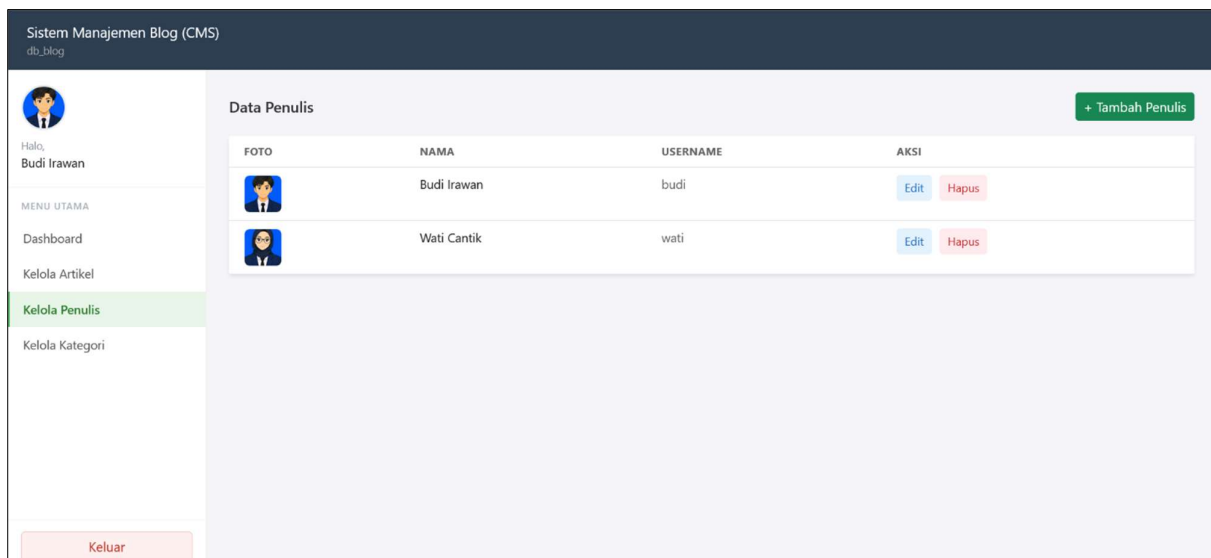
}}"
        nama_depan }}"
        style="width: 40px; height: 40px;
        object-fit: cover; border-radius: 6px;
        border: 1px solid #e9ecef;">
    </td>
    <td class="px-3 py-2" style="font-size: 13px;">
        {{ $item->nama_depan }} {{ $item->nama_belakang }}
    </td>
    <td class="px-3 py-2" style="font-size: 13px;
        color: #666666;">
        {{ $item->user_name }}
    </td>
    <td class="px-3 py-2">
        <div class="d-flex gap-2">
            <a href="{{ route('penulis.edit', $item->id)
            class="btn btn-sm"
            style="background-color: #e3f2fd;
            color: #1565c0; font-size: 12px;">
                Edit
            </a>
            <form action="{{ route('penulis.destroy',
            $item->id) }}"
                method="POST"
                onsubmit="return confirm('Hapus penulis
                ini?')">
                    @csrf
                    @method('DELETE')
                    <button type="submit" class="btn btn-sm"
                        style="background-color: #ffebee;
                        color: #c62828; font-size: 12px;">
                        Hapus
                    </button>
                </form>
            </div>
        </td>
    </tr>
    @empty
    <tr>
        <td colspan="4" class="px-3 py-4 text-center"
            style="font-size: 13px; color: #999999;
            font-style: italic;">
            Belum ada data penulis.
        </td>
    </tr>
    @endforelse
</tbody>
</table>
</div>
</div>
@endsection

```

Simpan file tersebut dengan menekan Ctrl+S. Buka browser dan akses:

```
http://localhost:8000/penulis
```

Browser akan menampilkan halaman daftar penulis beserta foto profil masing-masing seperti pada Gambar 10.7.



Gambar 10.7 Halaman daftar penulis

Langkah 14. Mengimplementasikan tambah penulis

Langkah ini mengimplementasikan fitur tambah penulis. Fitur ini mencakup form tambah dengan upload foto profil dan proses penyimpanan data ke database termasuk hashing password.

Mengisi method `create` dan `store` di `PenulisController`

Buka kembali file `app/Http/Controllers/PenulisController.php` dan isi method `create` dan `store` seperti berikut:

```
public function create()
{
    return view('penulis.create');
}

public function store(Request $request)
{
    $request->validate([
        'nama_depan'    => 'required|string|max:100',
        'nama_belakang' => 'required|string|max:100',
        'user_name'     => 'required|string|max:50|unique:penulis,user_name',
        'password'      => 'required|string|min:8',
        'foto'          => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
    ]);

    $namaFoto = 'default.png';

    if ($request->hasFile('foto')) {
        $file      = $request->file('foto');
        $namaFoto = time() . '_' . $file->getClientOriginalName();
        $file->storeAs('foto', $namaFoto, 'public');
    }

    Penulis::create([
        'nama_depan'    => $request->nama_depan,
        'nama_belakang' => $request->nama_belakang,
        'user_name'     => $request->user_name,
        'password'      => bcrypt($request->password),
        'foto'          => $namaFoto,
    ]);
}
```

```

return redirect()->route('penulis.index')
->with('sukses', 'Penulis berhasil ditambahkan.');
```

Penjelasan kode di atas:

- Nilai default foto** Jika pengguna tidak mengunggah foto, nilai `$namaFoto` tetap menggunakan `default.png` sebagai foto profil default.
- `$file->storeAs('foto', $namaFoto, 'public');`** Menyimpan foto ke folder `storage/app/public/foto/` dengan nama file yang sudah diberi timestamp di depannya untuk menghindari konflik nama file.
- `bcrypt($request->password)`** Menghasilkan hash bcrypt dari password yang dimasukkan sebelum disimpan ke database. Password tidak boleh disimpan dalam bentuk teks biasa.

Membuat View form tambah penulis

Buat file baru bernama `create.blade.php` di dalam folder `resources/views/penulis/`. Isi file tersebut dengan kode berikut:

```

@extends('layouts.app')

@section('title', 'Tambah Penulis')

@section('content')

<div class="d-flex justify-content-between align-items-center mb-3">
    <h6 class="fw-semibold mb-0" style="color: #333333;">Tambah Penulis</h6>
    <a href="{{ route('penulis.index') }}" class="btn btn-sm"
        style="background-color: #f0f0f0; color: #555555;">
        Kembali
    </a>
</div>

<div class="card border-0 shadow-sm">
    <div class="card-body p-4">
        <form action="{{ route('penulis.store') }}" method="POST"
            enctype="multipart/form-data">
            @csrf

            <div class="row g-3 mb-3">
                <div class="col-md-6">
                    <label for="nama_depan" class="form-label fw-semibold"
                        style="font-size: 13px;">
                        Nama Depan <span class="text-danger">*</span>
                    </label>
                    <input type="text"
                        class="form-control @error('nama_depan') is-invalid
@enderror"
                        id="nama_depan"
                        name="nama_depan"
                        value="{{ old('nama_depan') }}"
                        placeholder="Masukkan nama depan">
                    @error('nama_depan')
                        <div class="invalid-feedback">{{ $message }}</div>
                    @enderror
                </div>
                <div class="col-md-6">
                    <label for="nama_belakang" class="form-label fw-semibold"

```

```

        style="font-size: 13px;">
        Nama Belakang <span class="text-danger">*</span>
</label>
<input type="text"
        class="form-control @error('nama_belakang') is-invalid
@enderror"

        id="nama_belakang"
        name="nama_belakang"
        value="{{ old('nama_belakang') }}"
        placeholder="Masukkan nama belakang">
        @error('nama_belakang')
        <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>
</div>

<div class="mb-3">
    <label for="user_name" class="form-label fw-semibold"
        style="font-size: 13px;">
        Username <span class="text-danger">*</span>
    </label>
    <input type="text"
        class="form-control @error('user_name') is-invalid
@enderror"

        id="user_name"
        name="user_name"
        value="{{ old('user_name') }}"
        placeholder="Masukkan username">
        @error('user_name')
        <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>

<div class="mb-3">
    <label for="password" class="form-label fw-semibold"
        style="font-size: 13px;">
        Password <span class="text-danger">*</span>
    </label>
    <input type="password"
        class="form-control @error('password') is-invalid
@enderror"

        id="password"
        name="password"
        placeholder="Masukkan password (minimal 8 karakter)">
        @error('password')
        <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>

<div class="mb-4">
    <label for="foto" class="form-label fw-semibold"
        style="font-size: 13px;">
        Foto Profil
    </label>
    <input type="file"
        class="form-control @error('foto') is-invalid @enderror"
        id="foto"
        name="foto"
        accept="image/jpg,image/jpeg,image/png">
    <div class="form-text" style="font-size: 12px;">
        Format yang diizinkan: JPG, JPEG, PNG. Ukuran maksimal 2
        MB.
        Jika tidak diunggah, foto default akan digunakan.
    </div>

```



```

        @error('foto')
        <div class="invalid-feedback">{{ $message }}</div>
    @enderror
</div>

<div class="d-flex gap-2 justify-content-end">
    <a href="{{ route('penulis.index') }}" class="btn btn-sm"
        style="background-color: #f0f0f0; color: #555555;">
        Batal
    </a>
    <button type="submit" class="btn btn-sm btn-success">
        Simpan Data
    </button>
</div>

</form>
</div>
</div>
@endsection

```

Penjelasan bagian penting pada View di atas:

- enctype="multipart/form-data"** Atribut ini wajib ditambahkan pada form yang mengandung input file. Tanpa atribut ini, file yang diunggah tidak akan terkirim ke server.
- accept="image/jpg,image/jpeg,image/png"** Membatasi jenis file yang dapat dipilih di dialog file browser. Ini adalah validasi di sisi klien yang bersifat membantu namun tidak menggantikan validasi di sisi server.

Simpan file tersebut dengan menekan **Ctrl+S**. Buka browser dan akses:

```
http://localhost:8000/penulis/create
```

Browser akan menampilkan halaman form tambah penulis seperti pada Gambar 10.8.

The screenshot shows a web application interface for a CMS. On the left is a sidebar with a user profile (Budi Irawan) and a menu with options: MENU UTAMA, Dashboard, Kelola Artikel, **Kelola Penulis** (highlighted), and Kelola Kategori. The main content area is titled 'Tambah Penulis' and contains a form with the following fields: 'Nama Depan' and 'Nama Belakang' (both with placeholder text 'Masukkan nama depan/belakang'), 'Username' (placeholder 'Masukkan username'), 'Password' (placeholder 'Masukkan password (minimal 8 karakter)'), and 'Foto Profil' (a 'Browse...' button showing 'No file selected.'). Below the photo field is a note: 'Format yang diizinkan: JPG, JPEG, PNG. Ukuran maksimal 2 MB. Jika tidak diunggah, foto default akan digunakan.' At the bottom right of the form are two buttons: 'Batal' and 'Simpan Data'.

Gambar 10.8 Halaman form tambah penulis

Langkah 15. Mengimplementasikan edit penulis

Langkah ini mengimplementasikan fitur edit penulis. Fitur ini mencakup form edit yang sudah terisi data lama, penggantian foto profil yang bersifat opsional, dan penggantian password yang juga bersifat opsional.

Mengisi method edit dan update di PenulisController

Buka kembali file `app/Http/Controllers/PenulisController.php` dan isi method edit dan update seperti berikut:

```
public function edit(string $id)
{
    $penulis = Penulis::findOrFail($id);

    return view('penulis.edit', compact('penulis'));
}

public function update(Request $request, string $id)
{
    $penulis = Penulis::findOrFail($id);

    $request->validate([
        'nama_depan' => 'required|string|max:100',
        'nama_belakang' => 'required|string|max:100',
        'user_name' => 'required|string|max:50|unique:penulis,user_name,' .
    $id,
        'password' => 'nullable|string|min:8',
        'foto' => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
    ]);

    $data = [
        'nama_depan' => $request->nama_depan,
        'nama_belakang' => $request->nama_belakang,
        'user_name' => $request->user_name,
    ];

    if ($request->filled('password')) {
        $data['password'] = bcrypt($request->password);
    }

    if ($request->hasFile('foto')) {
        // Hapus foto lama jika bukan foto default
        if ($penulis->foto !== 'default.png') {
            Storage::disk('public')->delete('foto/' . $penulis->foto);
        }

        $file = $request->file('foto');
        $namaFoto = uniqid() . '.' . $file->getClientOriginalExtension();
        $file->storeAs('foto', $namaFoto, 'public');
        $data['foto'] = $namaFoto;
    }

    $penulis->update($data);

    return redirect()->route('penulis.index')
        ->with('sukses', 'Data penulis berhasil diperbarui.');
```

Penjelasan kode di atas:

- a. `$request->filled('password')` Method `filled()` memeriksa apakah field `password` terisi dan tidak kosong. Jika kosong, password lama di database tidak diubah. Ini memungkinkan pengguna memperbarui data penulis tanpa harus mengubah password setiap kali.
- b. **Penghapusan foto lama** Sebelum foto baru disimpan, foto lama dihapus dari storage menggunakan `Storage::disk('public')->delete('gambar/' . $artikel->gambar);` untuk menghindari penumpukan file yang tidak terpakai. Foto default tidak dihapus karena digunakan bersama oleh seluruh penulis yang belum mengunggah foto.
- c. **Array `$data`** Data yang akan diperbarui dikumpulkan terlebih dahulu dalam array `$data`. Password dan foto hanya ditambahkan ke array jika memenuhi kondisi masing-masing. Array ini kemudian dikirim sekaligus ke method `update()`.

Membuat View form edit penulis

Buat file baru bernama `edit.blade.php` di dalam folder `resources/views/penulis/`. Isi file tersebut dengan kode berikut:

```
@extends('layouts.app')

@section('title', 'Edit Penulis')

@section('content')

<div class="d-flex justify-content-between align-items-center mb-3">
    <h6 class="fw-semibold mb-0" style="color: #333333;">Edit Penulis</h6>
    <a href="{{ route('penulis.index') }}" class="btn btn-sm"
        style="background-color: #f0f0f0; color: #555555;">
        Kembali
    </a>
</div>

<div class="card border-0 shadow-sm">
    <div class="card-body p-4">
        <form action="{{ route('penulis.update', $penulis->id) }}"
method="POST"
        enctype="multipart/form-data">
            @csrf
            @method('PUT')

            <div class="row g-3 mb-3">
                <div class="col-md-6">
                    <label for="nama_depan" class="form-label fw-semibold"
                        style="font-size: 13px;">
                        Nama Depan <span class="text-danger">*</span>
                    </label>
                    <input type="text"
                        class="form-control @error('nama_depan') is-invalid
@enderror"
                        id="nama_depan"
                        name="nama_depan"
                        value="{{ old('nama_depan', $penulis->nama_depan) }}"
                        placeholder="Masukkan nama depan">
                    @error('nama_depan')
                        <div class="invalid-feedback">{{ $message }}</div>
                    @enderror
                </div>
                <div class="col-md-6">
```

```

        <label for="nama_belakang" class="form-label fw-semibold"
            style="font-size: 13px;">
            Nama Belakang <span class="text-danger">*</span>
        </label>
        <input type="text"
            class="form-control @error('nama_belakang') is-invalid
@enderror"
            id="nama_belakang"
            name="nama_belakang"
            value="{{ old('nama_belakang', $penulis-
>nama_belakang) }}"
            placeholder="Masukkan nama belakang">
        @error('nama_belakang')
        <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>
</div>

<div class="mb-3">
    <label for="user_name" class="form-label fw-semibold"
        style="font-size: 13px;">
        Username <span class="text-danger">*</span>
    </label>
    <input type="text"
        class="form-control @error('user_name') is-invalid
@enderror"
        id="user_name"
        name="user_name"
        value="{{ old('user_name', $penulis->user_name) }}"
        placeholder="Masukkan username">
    @error('user_name')
    <div class="invalid-feedback">{{ $message }}</div>
    @enderror
</div>

<div class="mb-3">
    <label for="password" class="form-label fw-semibold"
        style="font-size: 13px;">
        Password Baru
    </label>
    <input type="password"
        class="form-control @error('password') is-invalid
@enderror"
        id="password"
        name="password"
        placeholder="Kosongkan jika tidak ingin mengubah password">
    <div class="form-text" style="font-size: 12px;">
        Minimal 8 karakter. Kosongkan jika tidak ingin mengubah
password.
    </div>
    @error('password')
    <div class="invalid-feedback">{{ $message }}</div>
    @enderror
</div>

<div class="mb-4">
    <label for="foto" class="form-label fw-semibold"
        style="font-size: 13px;">
        Foto Profil
    </label>
    <div class="mb-2">
        
</div>
<input type="file"
  class="form-control @error('foto') is-invalid @enderror"
  id="foto"
  name="foto"
  accept="image/jpg,image/jpeg,image/png">
<div class="form-text" style="font-size: 12px;">
  Format yang diizinkan: JPG, JPEG, PNG. Ukuran maksimal 2
  MB.
  Kosongkan jika tidak ingin mengubah foto.
</div>
@error('foto')
  <div class="invalid-feedback">{{ $message }}</div>
@enderror
</div>

<div class="d-flex gap-2 justify-content-end">
  <a href="{{ route('penulis.index') }}" class="btn btn-sm"
    style="background-color: #f0f0f0; color: #555555;">
    Batal
  </a>
  <button type="submit" class="btn btn-sm btn-success">
    Simpan Perubahan
  </button>
</div>

</form>
</div>
</div>
@endsection

```

Penjelasan bagian penting pada View di atas:

- a. **Pratinjau foto profil** Foto profil penulis yang sedang diedit ditampilkan di atas input file sehingga pengguna dapat melihat foto saat ini sebelum memutuskan untuk menggantinya atau tidak.
- b. **Field password tanpa nilai default** Field password tidak diisi dengan nilai apapun karena password yang tersimpan di database sudah dalam bentuk hash dan tidak dapat dikembalikan ke bentuk aslinya.

Simpan file tersebut dengan menekan **Ctrl+S**. Buka browser dan akses halaman daftar penulis, lalu klik tombol **Edit** pada salah satu data. Browser akan menampilkan halaman form edit penulis seperti pada Gambar 10.9.

Gambar 10.9 Halaman form edit penulis

Langkah 16. Mengimplementasikan hapus penulis

Langkah ini mengimplementasikan fitur hapus penulis. Seperti pada fitur hapus kategori, fitur ini tidak memerlukan View baru karena tombol hapus sudah tersedia di halaman daftar penulis yang dibuat pada Langkah 13.

Mengisi method `destroy` di `PenulisController`

Buka kembali file `app/Http/Controllers/PenulisController.php` dan isi method `destroy` seperti berikut:

```
public function destroy(string $id)
{
    $penulis = Penulis::findOrFail($id);

    try {
        // Simpan nama foto dulu
        $foto = $penulis->foto;

        // Hapus data penulis
        $penulis->delete();

        // Setelah berhasil hapus database, baru hapus file
        if ($foto !== 'default.png') {
            Storage::disk('public')->delete('foto/' . $foto);
        }

        return redirect()->route('penulis.index')
            ->with('sukses', 'Data penulis berhasil dihapus.');
```

```
    } catch (\Exception $e) {

        return redirect()->route('penulis.index')
            ->with('gagal', 'Penulis tidak dapat dihapus karena masih
memiliki artikel.');
```

```
    }
}
```

```
}
```

Penjelasan kode di atas:

- a. **Penghapusan foto** Sebelum data penulis dihapus dari database, foto profil penulis dihapus terlebih dahulu dari storage. Foto default tidak dihapus karena digunakan bersama oleh seluruh penulis yang belum mengunggah foto.
- b. **try ... catch** Blok `catch` menangkap exception yang terjadi jika penghapusan gagal karena penulis masih memiliki artikel yang berelasi. Constraint `ON DELETE RESTRICT` pada foreign key `id_penulis` di tabel `artikel` mencegah penghapusan penulis yang masih memiliki artikel.

Berikut tampilan keseluruhan file `PenulisController.php` setelah seluruh method diisi:

```
<?php

namespace App\Http\Controllers;
use Illuminate\Support\Facades\Storage;
use Illuminate\Http\Request;
use App\Models\Penulis;

class PenulisController extends Controller
{
    public function index()
    {
        $penulis = Penulis::orderBy('nama_depan', 'asc')->get();

        return view('penulis.index', compact('penulis'));
    }

    public function create()
    {
        return view('penulis.create');
    }

    public function store(Request $request)
    {
        $request->validate([
            'nama_depan' => 'required|string|max:100',
            'nama_belakang' => 'required|string|max:100',
            'user_name' => 'required|string|max:50|unique:penulis,user_name',
            'password' => 'required|string|min:8',
            'foto' => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
        ]);

        $namaFoto = 'default.png';

        if ($request->hasFile('foto')) {
            $file = $request->file('foto');
            $namaFoto = uniqid() . '.' . $file->getClientOriginalExtension();
            $file->storeAs('foto', $namaFoto, 'public');
        }

        Penulis::create([
            'nama_depan' => $request->nama_depan,
            'nama_belakang' => $request->nama_belakang,
            'user_name' => $request->user_name,
            'password' => bcrypt($request->password),
            'foto' => $namaFoto,
        ]);
    }
}
```

```

    });

    return redirect()->route('penulis.index')
        ->with('sukses', 'Penulis berhasil ditambahkan.');
```

}

public function edit(string \$id)
{
 \$penulis = Penulis::findOrFail(\$id);

 return view('penulis.edit', compact('penulis'));
}

public function update(Request \$request, string \$id)
{
 \$penulis = Penulis::findOrFail(\$id);

 \$request->validate([
 'nama_depan' => 'required|string|max:100',
 'nama_belakang' => 'required|string|max:100',
 'user_name' => 'required|string|max:50|unique:penulis,user_name,' . \$id,
 'password' => 'nullable|string|min:8',
 'foto' => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
]);

 \$data = [
 'nama_depan' => \$request->nama_depan,
 'nama_belakang' => \$request->nama_belakang,
 'user_name' => \$request->user_name,
];

 if (\$request->filled('password')) {
 \$data['password'] = bcrypt(\$request->password);
 }

 if (\$request->hasFile('foto')) {
 // Hapus foto lama jika bukan foto default
 if (\$penulis->foto !== 'default.png') {
 Storage::disk('public')->delete('foto/' . \$penulis->foto);
 }

 \$file = \$request->file('foto');
 \$namaFoto = uniqid() . '.' . \$file->getClientOriginalExtension();
 \$file->storeAs('foto', \$namaFoto, 'public');
 \$data['foto'] = \$namaFoto;
 }

 \$penulis->update(\$data);

 return redirect()->route('penulis.index')
 ->with('sukses', 'Data penulis berhasil diperbarui.');
}

public function destroy(string \$id)
{
 \$penulis = Penulis::findOrFail(\$id);

 try {
 // Simpan nama foto dulu
 \$foto = \$penulis->foto;
 }
}


```

        // Hapus data penulis
        $penulis->delete();

        // Setelah berhasil hapus database, baru hapus file
        if ($foto !== 'default.png') {
            Storage::disk('public')->delete('foto/' . $foto);
        }

        return redirect()->route('penulis.index')
            ->with('sukses', 'Data penulis berhasil dihapus.');
```

```

    } catch (\Exception $e) {

        return redirect()->route('penulis.index')
            ->with('gagal', 'Penulis tidak dapat dihapus karena masih
memiliki artikel.');
```

```

    }
}
}

```

Simpan file tersebut dengan menekan Ctrl+S.

Untuk memverifikasi fitur hapus, buka browser dan akses halaman daftar penulis. Klik tombol **Hapus** pada salah satu penulis yang tidak memiliki artikel. Dialog konfirmasi akan muncul. Klik **OK** untuk melanjutkan penghapusan. Jika penulis masih memiliki artikel, pesan gagal akan ditampilkan di halaman daftar penulis.

CRUD Artikel

Langkah 17. Mengimplementasikan index artikel

Langkah ini mengimplementasikan halaman daftar artikel. Halaman ini menampilkan seluruh data artikel dalam bentuk tabel beserta gambar, judul, kategori, nama penulis, dan tanggal publikasi.

Mengisi method `index` di `ArtikelController`

Buka file `app/Http/Controllers/ArtikelController.php` dan tambahkan import Model serta isi method `index` seperti berikut:

```

<?php

namespace App\Http\Controllers;

use Illuminate\Support\Facades\Auth;
use Illuminate\Http\Request;
use App\Models\Artikel;
use App\Models\Penulis;
use App\Models\KategoriArtikel;

class ArtikelController extends Controller
{
    public function index()
    {
        $artikel = Artikel::with('penulis', 'kategori')
            ->orderBy('id', 'desc')
            ->get();

        return view('artikel.index', compact('artikel'));
    }
}

```

```

    public function create()
    {
        //
    }

    public function store(Request $request)
    {
        //
    }

    public function edit(string $id)
    {
        //
    }

    public function update(Request $request, string $id)
    {
        //
    }

    public function destroy(string $id)
    {
        //
    }
}

```

Penjelasan kode di atas:

- a. **Artikel::with('penulis', 'kategori')** Mengambil seluruh artikel beserta data penulis dan kategori yang berelasi menggunakan eager loading. Dengan eager loading, Laravel hanya menjalankan tiga query yaitu satu untuk artikel, satu untuk penulis, dan satu untuk kategori. Bukan satu query untuk setiap baris artikel.
- b. **->orderBy('id', 'desc')** Mengurutkan artikel dari yang paling baru berdasarkan kolom `id` secara menurun.

Membuat View index artikel

Buat folder `artikel` di dalam `resources/views/` melalui VS Code. Selanjutnya buat file baru bernama `index.blade.php` di dalam folder `artikel`. Isi file tersebut dengan kode berikut:

```

@extends('layouts.app')

@section('title', 'Kelola Artikel')

@section('content')

<div class="d-flex justify-content-between align-items-center mb-3">
    <h6 class="fw-semibold mb-0" style="color: #333333;">Data Artikel</h6>
    <a href="{{ route('artikel.create') }}" class="btn btn-sm btn-success">
        + Tambah Artikel
    </a>
</div>

<div class="card border-0 shadow-sm">
    <div class="card-body p-0">
        <table class="table table-hover mb-0">
            <thead>
                <tr style="background-color: #fafafa;">
                    <th class="px-3 py-2" style="font-size: 11px; color:
#666666;

```

```

                text-transform: uppercase; letter-spacing: 0.05em;">
                Gambar
            </th>
            <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                text-transform: uppercase; letter-spacing: 0.05em;">
                Judul
            </th>
            <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                text-transform: uppercase; letter-spacing: 0.05em;">
                Kategori
            </th>
            <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                text-transform: uppercase; letter-spacing: 0.05em;">
                Penulis
            </th>
            <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                text-transform: uppercase; letter-spacing: 0.05em;">
                Tanggal
            </th>
            <th class="px-3 py-2" style="font-size: 11px; color:
#666666;
                text-transform: uppercase; letter-spacing: 0.05em;">
                Aksi
            </th>
        </tr>
    </thead>
    <tbody>
        @forelse($artikel as $item)
            <tr>
                <td class="px-3 py-2">
                    judul }}"
                        style="width: 48px; height: 48px;
                        object-fit: cover; border-radius: 6px;
                        border: 1px solid #e9ecef;">
                </td>
                <td class="px-3 py-2" style="font-size: 13px;
                    max-width: 200px; overflow: hidden;
                    text-overflow: ellipsis; white-space: nowrap;">
                    {{ $item->judul }}
                </td>
                <td class="px-3 py-2" style="font-size: 13px;">
                    {{ $item->kategori->nama_kategori }}
                </td>
                <td class="px-3 py-2" style="font-size: 13px;">
                    {{ $item->penulis->nama_depan }}
                    {{ $item->penulis->nama_belakang }}
                </td>
                <td class="px-3 py-2"
                    style="font-size: 12px; color: #999999;">
                    {{ $item->hari_tanggal }}
                </td>
                <td class="px-3 py-2">
                    <div class="d-flex gap-2">
                        <a href="{{ route('artikel.edit', $item->id)
class="btn btn-sm"
                        style="background-color: #e3f2fd;
                        color: #1565c0; font-size: 12px;">

```

```

                                Edit
                                </a>
                                <form      action="{{      route('artikel.destroy',
$item->id) }}"
                                method="POST"
                                onsubmit="return  confirm('Hapus  artikel
ini?') ">
                                @csrf
                                @method('DELETE')
                                <button type="submit" class="btn btn-sm"
                                    style="background-color: #ffebee;
                                    color: #c62828; font-size: 12px;">
                                    Hapus
                                </button>
                                </form>
                                </div>
                            </td>
                        </tr>
                    @empty
                        <tr>
                            <td colspan="6" class="px-3 py-4 text-center"
                                style="font-size: 13px; color: #999999;
                                font-style: italic;">
                                Belum ada data artikel.
                            </td>
                        </tr>
                    @endforelse
                </tbody>
            </table>
        </div>
    </div>
@endsection

```

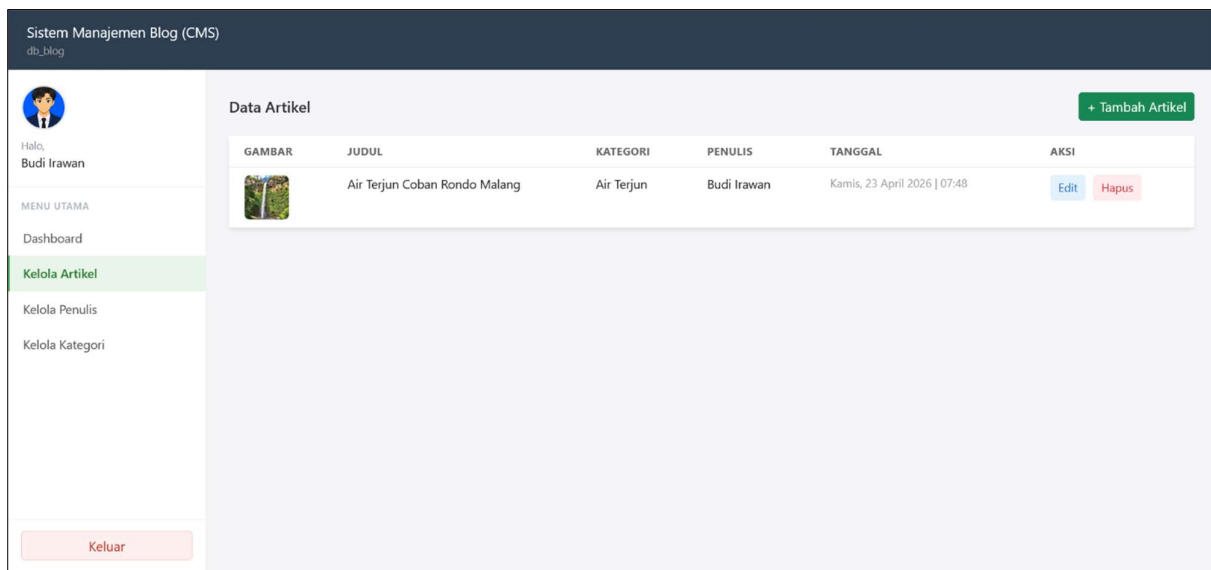
Penjelasan bagian penting pada View di atas:

- `$item->kategori->nama_kategori`** Mengakses data kategori yang berelasi dengan artikel melalui Eloquent Relationship yang sudah didefinisikan pada Langkah 3. Data ini tersedia karena sudah dimuat menggunakan eager loading di Controller.
- `$item->penulis->nama_depan`** Mengakses data penulis yang berelasi dengan artikel dengan cara yang sama.
- `max-width: 200px; overflow: hidden; text-overflow: ellipsis; white-space: nowrap`** Membatasi lebar kolom judul dan memotong teks yang terlalu panjang dengan tanda ellipsis agar tampilan tabel tetap rapi.

Simpan file tersebut dengan menekan **Ctrl+S**. Buka browser dan akses:

```
http://localhost:8000/artikel
```

Browser akan menampilkan halaman daftar artikel seperti pada Gambar 10.10.



Gambar 10.10 Halaman daftar artikel

Langkah 18. Mengimplementasikan tambah artikel

Langkah ini mengimplementasikan fitur tambah artikel. Fitur ini mencakup form tambah dengan dropdown kategori, upload gambar, serta pengisian kolom `id_penulis` secara otomatis dari data pengguna yang sedang login dan kolom `hari_tanggal` secara otomatis dari server.

Mengisi method `create` dan `store` di `ArtikelController`

Buka kembali file `app/Http/Controllers/ArtikelController.php`. Pastikan import berikut sudah ada di bagian atas file:

```
use App\Models\Artikel;
use App\Models\KategoriArtikel;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Storage;
```

Isi method `create` dan `store` seperti berikut:

```
public function create()
{
    $kategori = KategoriArtikel::orderBy('nama_kategori', 'asc')->get();

    return view('artikel.create', compact('kategori'));
}

public function store(Request $request)
{
    $request->validate([
        'judul'      => 'required|string|max:255',
        'isi'         => 'required|string',
        'id_kategori' => 'required|exists:kategori_artikel,id',
        'gambar'     => 'required|image|mimes:jpg,jpeg,png|max:2048',
    ]);

    $file          = $request->file('gambar');
    $namaFile      = uniqid() . '.' . $file->getClientOriginalExtension();
    $file->storeAs('gambar', $namaFile, 'public');

    Artikel::create([
        'judul'      => $request->judul,
```

```

        'isi'           => $request->isi,
        'id_penulis'    => Auth::user()->id,
        'id_kategori'   => $request->id_kategori,
        'gambar'        => $namaFile,
        'hari_tanggal' => now()->timezone('Asia/Jakarta')
                        ->locale('id')
                        ->isoFormat('dddd, D MMMM Y | HH:mm'),
    ];

    return redirect()->route('artikel.index')
        ->with('sukses', 'Artikel berhasil ditambahkan.');
```

Penjelasan kode di atas:

- Method `create()` mengambil data kategori dari database untuk mengisi dropdown di form tambah artikel. Data dikirim ke View menggunakan `compact()`. Data penulis tidak diperlukan karena `id_penulis` diambil langsung dari pengguna yang sedang login.
- `Auth::user()->id` Mengambil `id` pengguna yang sedang login dari session autentikasi. Dengan pendekatan ini, setiap artikel yang dibuat secara otomatis terhubung ke penulis yang sedang login tanpa perlu memilih dari dropdown.
- `uniqid()` Menghasilkan nama file unik berbasis waktu dalam format heksadesimal, misalnya `6834a2f1d3c8b.png`. Pendekatan ini menghindari konflik nama file tanpa menyertakan nama file asli yang mungkin mengandung spasi atau karakter khusus.
- `$file->storeAs('gambar', $namaFile, 'public')` Menyimpan file ke folder `storage/app/public/gambar/` menggunakan disk `public`.
- `now()->timezone('Asia/Jakarta')->locale('id')->isoFormat('dddd, D MMMM Y | HH:mm')` Menghasilkan string tanggal dan waktu saat artikel disimpan dalam format yang sudah ditentukan, misalnya `Kamis, 23 April 2026 | 07:48`. Timezone `Asia/Jakarta` digunakan agar waktu yang tersimpan sesuai dengan waktu lokal.

Membuat View form tambah artikel

Buat file baru bernama `create.blade.php` di dalam folder `resources/views/artikel/`. Isi file tersebut dengan kode berikut:

```

@extends('layouts.app')

@section('title', 'Tambah Artikel')

@section('content')

<div class="d-flex justify-content-between align-items-center mb-3">
    <h6 class="fw-semibold mb-0" style="color: #333333;">Tambah Artikel</h6>
    <a href="{{ route('artikel.index') }}" class="btn btn-sm"
        style="background-color: #f0f0f0; color: #555555;">
        Kembali
    </a>
</div>

<div class="card border-0 shadow-sm">
    <div class="card-body p-4">
```

```

<form action="{{ route('artikel.store') }}" method="POST"
    enctype="multipart/form-data">
    @csrf

    <div class="mb-3">
        <label for="judul" class="form-label fw-semibold"
            style="font-size: 13px;">
            Judul <span class="text-danger">*</span>
        </label>
        <input type="text"
            class="form-control @error('judul') is-invalid @enderror"
            id="judul"
            name="judul"
            value="{{ old('judul') }}"
            placeholder="Masukkan judul artikel">
        @error('judul')
            <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>

    <div class="mb-3">
        <label for="id_kategori" class="form-label fw-semibold"
            style="font-size: 13px;">
            Kategori <span class="text-danger">*</span>
        </label>
        <select class="form-select @error('id_kategori') is-invalid
@enderror"
            id="id_kategori"
            name="id_kategori">
            <option value="">Pilih Kategori</option>
            @foreach($kategori as $item)
                <option value="{{ $item->id }}"
                    {{ old('id_kategori') == $item->id ? 'selected' :
' ' }}>
                    {{ $item->nama_kategori }}
                </option>
            @endforeach
        </select>
        @error('id_kategori')
            <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>

    <div class="mb-3">
        <label for="isi" class="form-label fw-semibold"
            style="font-size: 13px;">
            Isi Artikel <span class="text-danger">*</span>
        </label>
        <textarea class="form-control @error('isi') is-invalid
@enderror"
            id="isi"
            name="isi"
            rows="6"
            placeholder="Masukkan isi artikel">{{ old('isi')
}}</textarea>
        @error('isi')
            <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>

    <div class="mb-4">
        <label for="gambar" class="form-label fw-semibold"
            style="font-size: 13px;">
            Gambar <span class="text-danger">*</span>

```

```

        </label>
        <input type="file"
            class="form-control @error('gambar') is-invalid @enderror"
            id="gambar"
            name="gambar"
            accept="image/jpg,image/jpeg,image/png">
        <div class="form-text" style="font-size: 12px;">
            Format yang diizinkan: JPG, JPEG, PNG. Ukuran maksimal 2
MB.
        </div>
        @error('gambar')
            <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>

    <div class="d-flex gap-2 justify-content-end">
        <a href="{{ route('artikel.index') }}" class="btn btn-sm"
            style="background-color: #f0f0f0; color: #555555;">
            Batal
        </a>
        <button type="submit" class="btn btn-sm btn-success">
            Simpan Data
        </button>
    </div>

</form>
</div>
</div>
@endsection

```

Penjelasan bagian penting pada View di atas:

- old('id_kategori') == \$item->id ? 'selected' : ''** Mempertahankan pilihan dropdown kategori yang sudah dipilih pengguna jika validasi gagal dan halaman form ditampilkan kembali. Tanpa ini, dropdown akan kembali ke pilihan default setiap kali validasi gagal.
- enctype="multipart/form-data"** Wajib ditambahkan pada form yang mengandung input file agar file yang diunggah dapat dikirim ke server.

Simpan file tersebut dengan menekan **Ctrl+S**. Buka browser dan akses:

```
http://localhost:8000/artikel/create
```

Browser akan menampilkan halaman form tambah artikel seperti pada Gambar 10.11.

Gambar 10.11 Halaman form tambah artikel

Langkah 19. Mengimplementasikan edit artikel

Langkah ini mengimplementasikan fitur edit artikel. Fitur ini mencakup form edit yang sudah terisi data lama, dropdown kategori yang menampilkan pilihan yang sesuai dengan data artikel yang sedang diedit, serta penggantian gambar yang bersifat opsional. Kolom `id_penulis` tidak dapat diubah saat edit. Artikel tetap terhubung ke penulis yang pertama kali membuatnya.

Mengisi method `edit` dan `update` di `ArtikelController`

Buka kembali file `app/Http/Controllers/ArtikelController.php` dan isi method `edit` dan `update` seperti berikut:

```
public function edit(string $id)
{
    $artikel = Artikel::findOrFail($id);
    $kategori = KategoriArtikel::orderBy('nama_kategori', 'asc')->get();

    return view('artikel.edit', compact('artikel', 'kategori'));
}

public function update(Request $request, string $id)
{
    $artikel = Artikel::findOrFail($id);

    $request->validate([
        'judul' => 'required|string|max:255',
        'isi' => 'required|string',
        'id_kategori' => 'required|exists:kategori_artikel,id',
        'gambar' => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
    ]);

    $data = [
        'judul' => $request->judul,
        'isi' => $request->isi,
        'id_kategori' => $request->id_kategori,
```

```

];

if ($request->hasFile('gambar')) {
    Storage::disk('public')->delete('gambar/' . $artikel->gambar);

    $file      = $request->file('gambar');
    $namaFile = uniqid() . '.' . $file->getClientOriginalExtension();
    $file->storeAs('gambar', $namaFile, 'public');

    $data['gambar'] = $namaFile;
}

$artikel->update($data);

return redirect()->route('artikel.index')
    ->with('sukses', 'Artikel berhasil diperbarui.');
```

Penjelasan kode di atas:

- Method `edit()` mengambil data artikel yang sedang diedit beserta seluruh data kategori untuk mengisi dropdown. Data penulis tidak diperlukan karena `id_penulis` tidak dapat diubah saat edit.
- Kolom `id_penulis` tidak dimasukkan ke array `$data`** Ini memastikan penulis artikel tidak berubah meskipun pengguna yang sedang login berbeda dengan penulis artikel yang sedang diedit.
- Kolom `hari_tanggal` tidak dimasukkan ke array `$data`** Tanggal publikasi artikel tetap menggunakan tanggal pertama kali artikel dibuat dan tidak berubah saat artikel diedit.
- `$file->storeAs('gambar', $namaFile, 'public')` Menyimpan file ke folder `storage/app/public/gambar/` menggunakan disk `public`.
- Penghapusan gambar lama** Saat gambar baru diunggah, gambar lama dihapus dari storage terlebih dahulu menggunakan `Storage::disk('public')->delete('gambar/' . $artikel->gambar);` untuk menghindari penumpukan file yang tidak terpakai.

Membuat View form edit artikel

Buat file baru bernama `edit.blade.php` di dalam folder `resources/views/artikel/`. Isi file tersebut dengan kode berikut:

```

@extends('layouts.app')

@section('title', 'Edit Artikel')

@section('content')

<div class="d-flex justify-content-between align-items-center mb-3">
    <h6 class="fw-semibold mb-0" style="color: #333333;">Edit Artikel</h6>
    <a href="{{ route('artikel.index') }}" class="btn btn-sm"
        style="background-color: #f0f0f0; color: #555555;">
        Kembali
    </a>
</div>

<div class="card border-0 shadow-sm">
```

```

<div class="card-body p-4">
    <form action="{{ route('artikel.update', $artikel->id) }}"
method="POST"
    enctype="multipart/form-data">
        @csrf
        @method('PUT')

        <div class="mb-3">
            <label for="judul" class="form-label fw-semibold"
                style="font-size: 13px;">
                Judul <span class="text-danger">*</span>
            </label>
            <input type="text"
                class="form-control @error('judul') is-invalid @enderror"
                id="judul"
                name="judul"
                value="{{ old('judul', $artikel->judul) }}"
                placeholder="Masukkan judul artikel">
            @error('judul')
                <div class="invalid-feedback">{{ $message }}</div>
            @enderror
        </div>

        <div class="mb-3">
            <label for="id_kategori" class="form-label fw-semibold"
                style="font-size: 13px;">
                Kategori <span class="text-danger">*</span>
            </label>
            <select class="form-select @error('id_kategori') is-invalid
@enderror"
                id="id_kategori"
                name="id_kategori">
                <option value="">Pilih Kategori</option>
                @foreach($kategori as $item)
                    <option value="{{ $item->id }}"
                        {{ old('id_kategori', $artikel->id_kategori) ==
$item->id ? 'selected' : '' }}>
                        {{ $item->nama_kategori }}
                    </option>
                @endforeach
            </select>
            @error('id_kategori')
                <div class="invalid-feedback">{{ $message }}</div>
            @enderror
        </div>

        <div class="mb-3">
            <label for="isi" class="form-label fw-semibold"
                style="font-size: 13px;">
                Isi Artikel <span class="text-danger">*</span>
            </label>
            <textarea class="form-control @error('isi') is-invalid
@enderror"
                id="isi"
                name="isi"
                rows="6"
                placeholder="Masukkan isi artikel">{{ old('isi', $artikel-
>isi) }}</textarea>
            @error('isi')
                <div class="invalid-feedback">{{ $message }}</div>
            @enderror
        </div>

        <div class="mb-4">

```

```

        <label for="gambar" class="form-label fw-semibold"
            style="font-size: 13px;">
            Gambar
        </label>
        <div class="mb-2">
            
        </div>
        <input type="file"
            class="form-control @error('gambar') is-invalid @enderror"
            id="gambar"
            name="gambar"
            accept="image/jpeg,image/jpeg,image/png">
        <div class="form-text" style="font-size: 12px;">
            Format yang diizinkan: JPG, JPEG, PNG. Ukuran maksimal 2
            MB.
            Kosongkan jika tidak ingin mengubah gambar.
        </div>
        @error('gambar')
            <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>

    <div class="d-flex gap-2 justify-content-end">
        <a href="{{ route('artikel.index') }}" class="btn btn-sm"
            style="background-color: #f0f0f0; color: #555555;">
            Batal
        </a>
        <button type="submit" class="btn btn-sm btn-success">
            Simpan Perubahan
        </button>
    </div>

    </form>
</div>
</div>
@endsection

```

Penjelasan bagian penting pada View di atas:

- Penjelasan bagian penting pada View di atas:
- `old('id_kategori', $artikel->id_kategori) == $item->id` Argumen kedua pada helper `old()` menyediakan nilai default berupa `id_kategori` dari artikel yang sedang diedit. Saat halaman pertama kali dibuka, dropdown menampilkan kategori yang sudah terpilih. Jika validasi gagal, dropdown mempertahankan pilihan terakhir pengguna.
- `enctype="multipart/form-data"` Wajib ditambahkan pada form yang mengandung input file agar file yang diunggah dapat dikirim ke server.
- Pratinjau gambar** Gambar artikel yang sedang diedit ditampilkan di atas input file sehingga pengguna dapat melihat gambar saat ini sebelum memutuskan untuk menggantinya.

Simpan file tersebut dengan menekan **Ctrl+S**. Buka browser dan akses halaman daftar artikel, lalu klik tombol **Edit** pada salah satu data. Browser akan menampilkan halaman form edit artikel seperti pada Gambar 10.12.

The screenshot shows a web application titled 'Sistem Manajemen Blog (CMS)' with a sidebar menu containing 'Halo, Wati Cantik', 'MENU UTAMA', 'Dashboard', 'Kelola Artikel' (highlighted), 'Kelola Penulis', and 'Kelola Kategori'. The main content area is titled 'Edit Artikel' and contains the following form fields:

- Judul ***: A text input field containing 'Air Terjun Coban Rondo Malang'.
- Kategori ***: A dropdown menu with 'Air Terjun' selected.
- Isi Artikel ***: A text area containing a paragraph about Coban Rondo.
- Gambar**: A section with a small image preview and a 'Browse...' button. Below it, it says 'No file selected.' and 'Format yang diizinkan: JPG, JPEG, PNG. Ukuran maksimal 2 MB. Kosongkan jika tidak ingin mengubah gambar.'

At the bottom right of the form are two buttons: 'Batal' and 'Simpan Perubahan'.

Gambar 10.12 Halaman form edit artikel

Langkah 20. Mengimplementasikan hapus artikel

Langkah ini mengimplementasikan fitur hapus artikel. Seperti pada fitur hapus kategori dan penulis, fitur ini tidak memerlukan View baru karena tombol hapus sudah tersedia di halaman daftar artikel yang dibuat pada Langkah 17.

Mengisi method **destroy** di **ArtikelController**

Buka kembali file `app/Http/Controllers/ArtikelController.php` dan isi method `destroy` seperti berikut:

```
public function destroy(string $id)
{
    $artikel = Artikel::findOrFail($id);

    try {
        Storage::disk('public')->delete('gambar/' . $artikel->gambar);

        $artikel->delete();

        return redirect()->route('artikel.index')
            ->with('sukses', 'Artikel berhasil dihapus.');
```

```
    } catch (\Exception $e) {
        return redirect()->route('artikel.index')
            ->with('gagal', 'Artikel gagal dihapus.');
```

```
    }
}
```

Penjelasan kode di atas:

Penghapusan gambar Gambar artikel dihapus dari storage sebelum data artikel dihapus dari database. Berbeda dengan foto profil penulis, gambar artikel tidak memiliki gambar default sehingga pengecekan tidak diperlukan. Gambar selalu dihapus saat artikel dihapus.

Berikut tampilan keseluruhan file `ArtikelController.php` setelah seluruh method diisi:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Storage;
use Illuminate\Http\Request;
use App\Models\Artikel;
use App\Models\Penulis;
use App\Models\KategoriArtikel;

class ArtikelController extends Controller
{
    public function index()
    {
        $artikel = Artikel::with('penulis', 'kategori')
            ->orderBy('id', 'desc')
            ->get();

        return view('artikel.index', compact('artikel'));
    }

    public function create()
    {
        $kategori = KategoriArtikel::orderBy('nama_kategori', 'asc')->get();

        return view('artikel.create', compact('kategori'));
    }

    public function store(Request $request)
    {
        $request->validate([
            'judul'      => 'required|string|max:255',
            'isi'         => 'required|string',
            'id_kategori' => 'required|exists:kategori_artikel,id',
            'gambar'      => 'required|image|mimes:jpg,jpeg,png|max:2048',
        ]);

        $file      = $request->file('gambar');
        $namaFile = uniqid() . '.' . $file->getClientOriginalExtension();
        $file->storeAs('gambar', $namaFile, 'public');

        Artikel::create([
            'judul'      => $request->judul,
            'isi'         => $request->isi,
            'id_penulis' => Auth::user()->id,
            'id_kategori' => $request->id_kategori,
            'gambar'      => $namaFile,
            'hari_tanggal' => now()->timezone('Asia/Jakarta')
                ->locale('id')
                ->isoFormat('dddd, D MMMM Y | HH:mm'),
        ]);

        return redirect()->route('artikel.index')
            ->with('sukses', 'Artikel berhasil ditambahkan.');
```

```

    }

    public function edit(string $id)
    {
        $artikel = Artikel::findOrFail($id);
        $kategori = KategoriArtikel::orderBy('nama_kategori', 'asc')->get();

        return view('artikel.edit', compact('artikel', 'kategori'));
    }

    public function update(Request $request, string $id)
    {
        $artikel = Artikel::findOrFail($id);

        $request->validate([
            'judul' => 'required|string|max:255',
            'isi' => 'required|string',
            'id_kategori' => 'required|exists:kategori_artikel,id',
            'gambar' => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
        ]);

        $data = [
            'judul' => $request->judul,
            'isi' => $request->isi,
            'id_kategori' => $request->id_kategori,
        ];

        if ($request->hasFile('gambar')) {
            Storage::disk('public')->delete('gambar/' . $artikel->gambar);

            $file = $request->file('gambar');
            $namaFile = uniqid() . '.' . $file->getClientOriginalExtension();
            $file->storeAs('gambar', $namaFile, 'public');

            $data['gambar'] = $namaFile;
        }

        $artikel->update($data);

        return redirect()->route('artikel.index')
            ->with('sukses', 'Artikel berhasil diperbarui.');
```

```

    }

    public function destroy(string $id)
    {
        $artikel = Artikel::findOrFail($id);

        try {
            Storage::disk('public')->delete('gambar/' . $artikel->gambar);

            $artikel->delete();

            return redirect()->route('artikel.index')
                ->with('sukses', 'Artikel berhasil dihapus.');
```

```

        } catch (\Exception $e) {
            return redirect()->route('artikel.index')
                ->with('gagal', 'Artikel gagal dihapus.');
```

```

        }
    }
}

```

Simpan file tersebut dengan menekan Ctrl+S.

Untuk memverifikasi fitur hapus, buka browser dan akses halaman daftar artikel. Klik tombol **Hapus** pada salah satu artikel. Dialog konfirmasi akan muncul. Klik **OK** untuk melanjutkan penghapusan. Pastikan gambar artikel juga terhapus dari folder `storage/app/public/gambar/` setelah penghapusan berhasil.

Finalisasi

Langkah 21. Verifikasi seluruh fitur di browser

Pastikan development server masih berjalan dan MySQL XAMPP sudah aktif. Jika server sudah dihentikan, jalankan kembali dengan perintah berikut:

```
php artisan serve
```

Lakukan pengujian secara berurutan untuk memastikan seluruh fitur berjalan dengan benar.

Pengujian 1. Autentikasi

Buka browser dan akses:

```
http://localhost:8000
```

Browser akan diarahkan otomatis ke halaman login. Masukkan username dan password yang benar. Setelah login berhasil, browser akan diarahkan ke halaman dashboard yang menampilkan sapaan, panduan, dan waktu login.

Pengujian 2. Navigasi Sidebar

Dari halaman dashboard, klik setiap menu di sidebar secara bergantian dan pastikan:

- Menu **Kelola Artikel** menampilkan halaman daftar artikel
- Menu **Kelola Penulis** menampilkan halaman daftar penulis
- Menu **Kelola Kategori** menampilkan halaman daftar kategori
- Menu yang sedang aktif ditampilkan dengan warna hijau di sidebar

Pengujian 3. CRUD Kategori Artikel

Akses menu **Kelola Kategori** dan lakukan pengujian berikut:

- Klik tombol **Tambah Kategori**, isi form, lalu simpan. Pastikan data baru muncul di tabel dan pesan sukses ditampilkan.
- Klik tombol **Edit** pada salah satu kategori, ubah data, lalu simpan. Pastikan data berhasil diperbarui.
- Coba hapus kategori yang masih memiliki artikel. Pastikan pesan gagal ditampilkan.
- Hapus kategori yang tidak memiliki artikel. Pastikan data berhasil dihapus.

Pengujian 4. CRUD Penulis

Akses menu **Kelola Penulis** dan lakukan pengujian berikut:

- Klik tombol **Tambah Penulis**, isi form tanpa mengunggah foto, lalu simpan. Pastikan foto default ditampilkan di tabel.
- Tambah penulis baru dengan mengunggah foto. Pastikan foto tampil dengan benar di tabel.

- Klik tombol **Edit** pada salah satu penulis, ubah data tanpa mengubah password dan foto, lalu simpan. Pastikan password dan foto lama tidak berubah.
- Edit penulis dengan mengunggah foto baru. Pastikan foto lama terganti dan foto baru tampil di tabel.
- Coba hapus penulis yang masih memiliki artikel. Pastikan pesan gagal ditampilkan.
- Hapus penulis yang tidak memiliki artikel. Pastikan data dan foto profil terhapus.

Pengujian 5. CRUD Artikel

Akses menu **Kelola Artikel** dan lakukan pengujian berikut:

- Klik tombol **Tambah Artikel**, isi seluruh field termasuk gambar, lalu simpan. Pastikan artikel baru muncul di tabel dengan kolom `hari_tanggal` terisi otomatis dalam format yang benar.
- Klik tombol **Edit** pada salah satu artikel, ubah judul dan isi tanpa mengganti gambar, lalu simpan. Pastikan gambar lama tetap tampil dan tanggal tidak berubah.
- Edit artikel dengan mengunggah gambar baru. Pastikan gambar lama terganti dan gambar baru tampil di tabel.
- Hapus salah satu artikel. Pastikan data dan gambar artikel terhapus dari storage.

Pengujian 6. Validasi Input

Pada setiap form tambah dan edit, coba kirim form dalam keadaan kosong atau dengan data yang tidak valid. Pastikan pesan error ditampilkan di bawah setiap field yang tidak valid dan nilai yang sudah diisi sebelumnya tetap tersimpan di form.

Pengujian 7. Logout

Klik tombol **Keluar** di sidebar. Pastikan browser diarahkan kembali ke halaman login. Coba akses halaman dashboard secara langsung:

<http://localhost:8000/dashboard>

Pastikan browser diarahkan otomatis ke halaman login karena session sudah dihapus.

Penutup

Bab ini membahas pengembangan proyek `aplikasi-blog` dari sistem autentikasi sederhana yang dibangun pada Bab 9 menjadi sistem manajemen konten yang lengkap. Eloquent ORM digunakan sebagai antarmuka utama untuk berinteraksi dengan database menggantikan query SQL manual yang telah digunakan pada bab-bab sebelumnya. Relasi antar tabel didefinisikan menggunakan Eloquent Relationship sehingga data dari tabel yang berelasi dapat diakses langsung sebagai properti objek tanpa perlu menulis query JOIN secara manual.

Operasi CRUD diimplementasikan untuk tiga entitas sekaligus yaitu kategori artikel, penulis, dan artikel menggunakan Resource Controller yang mengikuti konvensi RESTful. Setiap Controller memiliki tanggung jawab yang jelas dan terpisah sesuai dengan prinsip single responsibility. Tampilan dibangun menggunakan Bootstrap 5 dengan layout utama Blade yang diwarisi oleh seluruh

View sehingga struktur halaman konsisten tanpa duplikasi kode HTML. Upload gambar ditangani menggunakan Laravel Storage dengan folder terpisah untuk foto profil penulis dan gambar artikel. Validasi input diterapkan di sisi server menggunakan fitur validasi bawaan Laravel, dan notifikasi hasil operasi ditampilkan menggunakan session flash message mengikuti pola PRG.

Seluruh halaman dilindungi menggunakan middleware `auth` yang dibangun pada Bab 9 sehingga hanya pengguna yang sudah login yang dapat mengakses dan mengelola data.